arXiv:2407.13278v3 [cs.LG] 4 May 2026

Deep Time Series Models: A Comprehensive Survey and Benchmark

Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Chen Wang, Mingsheng Long, Jianmin Wang

Abstract—Time series, characterized by a sequence of data points organized in a discrete-time order, are ubiquitous in real-world scenarios. Unlike other data modalities, time series present unique challenges in learning and modeling due to their intricate and dynamic nature, including the entanglement of nonlinear patterns and time-variant trends. Recent years have witnessed remarkable breakthroughs in time series analysis, with techniques shifting from traditional statistical methods to contemporary deep learning models. In this paper, we delve into the design of deep time series models across various analysis tasks and review the existing literature from two perspectives: basic modules and model architectures. Further, we develop and release Time Series Library (TSLib) as a fair benchmark of deep time series models for diverse analysis tasks. TSLib implements 41 prominent models, including both small- and large-scale time series models, covers 30 datasets from different domains, and supports 5 prevalent analysis tasks. Based on TSLib, we evaluate 16 popular deep time series models and 6 advanced time series foundation models. Empirical findings indicate that models with specific structures are apt only at distinct analytical tasks, providing insights for research and adoption of deep time series models. Code and datasets are available at https://github.com/thuml/Time-Series-Library.

Index Terms—Time series analysis, Deep time series models, Time series foundation models, Survey, Benchmark

1 Introduction

Time series refers to a sequence of data points indexed in a discrete-time order [1], [2], which are omnipresent in real-world applications, such as financial risk assessment, energy sustainability, and weather forecasting. Driven by the increasing availability of vast amounts of time series data across various domains, the community of time series analysis has witnessed tremendous advancements. Compared to image and text data, which have objectively prescribed syntax or intuitive patterns, the semantic information of time series data is primarily derived from the temporal variation [3]. This presents significant challenges in understanding such data, notably in identifying sequential dependencies, trends, seasonal patterns, and complicated dynamics. Consequently, analyzing time series data necessitates sophisticated methods to extract its meaningful temporal representations.

Time series analysis has long stood as a pivotal research area, owing to its indispensable role in real-world applications [4], [5], [6]. It involves scrutinizing temporal variations in sequential data to uncover underlying patterns, thereby facilitating accurate predictions and informed decision-making. A fundamental aspect of this process lies in identifying intricate temporal dependencies and variable correlations inherent in the data. By capturing these entangled dependencies, time series models reveal underlying dynamics and support a broad spectrum of downstream tasks, including forecasting, classification, imputation, and anomaly detection.

Traditional approaches, such as Autoregressive Integrated Moving Average (ARIMA) [1], Exponential Smoothing, and Spectral Analysis [7], have long served as stalwart tools for analyzing time series data. Grounded in statistical methodologies, these methods have been instrumental in uncovering patterns, trends, and seasonality within temporal variations. However, their rigid assumptions of linearity and stationarity limit their application to dynamic and evolving data flows, making them inadequate for capturing the intricate nonlinear relationships and long-term dependencies present in real-world time series data.

Deep models have garnered wide attention and achieved remarkable performance across various domains, including natural language processing (NLP) [16], [17], computer vision (CV) [18], [19], and recommender systems [20]. In recent years, deep learning models [3], [21], [22], [23], [24] have proven highly effective in capturing the intricate dependencies within time series data, making them a powerful tool for time series analysis over traditional statistical methods. More recently, Transformer models with attention mechanisms, originally developed for natural language processing tasks, have exhibited unprecedented capability in processing large-scale data [17] and have also been adapted for learning time series data. These architectures offer the advantage of selectively focusing on different parts of the input sequence, allowing for more nuanced discovery of both temporal and variable dependencies in time series.

While various time series models designed for different analysis tasks have emerged in recent years, there is a lack of a comprehensive overview of existing methods, covering both tasks and models. Previous reviews focus exclusively on either a specific model architecture or an analysis task. For example, [8], [9], [10], [11] each reviews deep learning methods for specific time series analysis tasks but fails to include advanced architecture such as Transformers. Several surveys [12], [13] provide up-to-date reviews for time series analysis focusing on specific deep learning architectures

- *The authors are with the School of Software, BNRist, Tsinghua University, Beijing 100084, China. E-mail: wangyuxu22@mails.tsinghua.edu.cn.*
- *Yuxuan Wang, Haixu Wu and Jiaxiang Dong contributed equally.*
- *Corresponding authors: Mingsheng Long, Jianmin Wang.*

TABLE 1
Difference between our work and other related surveys. We present a systematic review of tasks and models, and offer an extensive benchmark.

Survey	Analysis Task				Model Architecture					Benchmark
	Forecasting	Classification	Imputation	Anomaly Detection	MLP	CNN	RNN	GNN	Transformer	
Fawaz *et al.* (2019) [8]		✓			✓	✓	✓			
Braei *et al.* (2020) [9]				✓	✓	✓	✓			
Torres *et al.* (2021) [10]	✓					✓	✓	✓		
García *et al.* (2021) [11]				✓		✓	✓	✓		
Wen *et al.* (2022) [12]	✓	✓		✓					✓	
Jin *et al.* (2023) [13]	✓	✓	✓	✓				✓		
Shao *et al.* (2023) [14]	✓				✓	✓	✓	✓	✓	✓
Qiu *et al.* (2024) [15]	✓				✓	✓	✓	✓	✓	✓
This Survey	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

(e.g., Graph Neural Networks and Transformers). Recently, BasicTS [14] and TFB [15] introduce forecasting benchmarks that enable an unbiased evaluation of existing approaches but do not provide an overview of the architectural design of those deep models.

In this survey, we try to provide a unique review of deep time series models for researchers and practitioners, starting from the basic modules to modern architectures. To foster practical applications, a time series benchmark is offered for a fair evaluation and to identify the effective scope of existing models. This survey is organized as follows. Section 2 provides the background concepts of time series analysis. Section 3 introduces the basic modules that are widely utilized in prevalent deep time series models. Section 4 reviews existing deep time series models in terms of the architecture design. Section 5 discusses recent processes in time series foundation models. Section 6 introduces an easy-to-use open-source benchmark, Time Series Library (TSLib), along with extensive experimental comparisons and detailed analyses. Section 7 provides a brief discussion of future research directions while Section 8 summarizes this survey.

2 Preliminaries

2.1 Time Series

Time series is a sequence of T observations ordered by time, which can be denoted as $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_T\} \in \mathbb{R}^{T \times C}$, where $\mathbf{x}_t \in \mathbb{R}^C$ represents the observed values at timestamp t and C is the number of variables. Note that spatiotemporal data (e.g. videos or weather records) can also be viewed as a special type of time series, where the "variate dimension" of the above notion corresponds to spatial pixels [25] in a video. However, previous research on spatiotemporal data primarily relies on highly structured spatial information [26], which limits their applications to general variates, like heterogeneous variates with distinct semantic meanings. In this paper, we focus on the more generic formalization, where we do not assume any spatial structure inside the variate dimension. Under this definition, the essence of time series analysis is to capture and utilize the temporal dependencies and inter-variable correlations within the observations.

Temporal Dependency Given the inherent sequential nature of time series data, one evident technological paradigm is to capture the temporal dependence of a set of historical data. Essentially, temporal dependencies refer to the intricate connections that manifest among distinct time points or subseries. Traditional statistical methods, such as those based on autoregressive principles represented by ARIMA [1], have been widely studied to model these dependencies. Owing to their simplicity and interpretability, these statistical methods retain prominence in scenarios where the underlying temporal dynamics exhibit limited complexity.

Variate Correlation Beyond temporal dependencies, comprehending the correlations among distinct variables is paramount for analyzing multivariate time series, particularly in high-dimensional contexts. These correlations encapsulate the complex interactions and associations among multiple measurements as they evolve over time. Elucidating these relationships provides valuable insights into the underlying dynamics and latent processes. Conventional approaches, exemplified by VAR [27], extend the concept of autoregression to multiple variables, thereby enabling the capture of inter-variable relationships across time. However, VAR models represent each variable as a linear combination of lagged values of all other variables, which inherently limits their ability to capture misaligned or nonlinear relationships.

The aforementioned fundamental elements of time series present considerable challenges for constructing practical analysis models. While traditional statistical methods offer foundational theoretical insights, their inherent reliance on presumptions of the data, such as linearity and stationarity, often restricts their applicability in intricate real-world scenarios. Recently, deep learning models have emerged as powerful alternatives for time series modeling. Their capacity for representation learning and scalability to big data uniquely positions them for high-dimensional and nonlinear time series data, effectively fulfilling diverse analysis tasks.

2.2 Time Series Analysis Tasks

Based on underlying patterns and trends within time series data, time series analysis encompasses various downstream applications, including forecasting [28], [29], imputation [30], [31], [32], classification [8], [33], and anomaly detection [5], [34], each finding distinct applications in diverse domains, as illustrated in Figure 1.

Forecasting is a fundamental task in time series analysis that requires models to uncover temporal dependencies and dynamic patterns. By capturing the relationships between past and future data, the forecasting model aims to predict future values or trends of the input series. Missing data

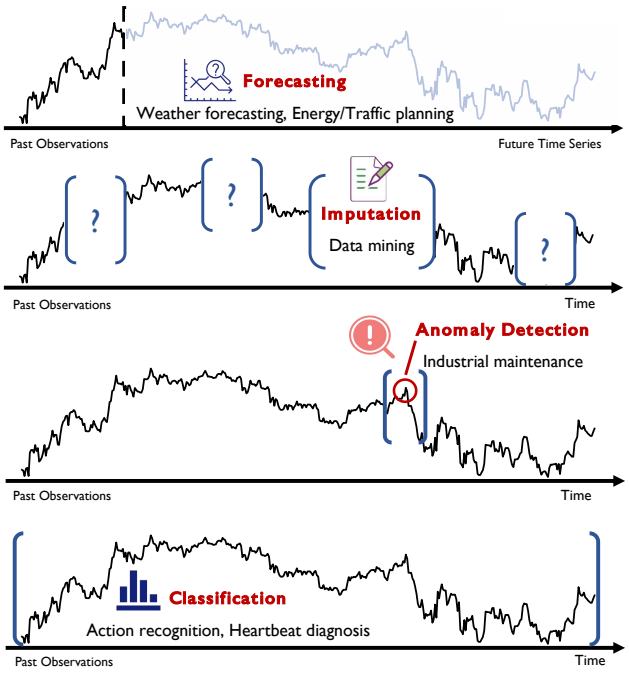


Fig. 1. Schematic illustration of different time series analysis tasks.

due to sensor failures, data corruption, or absent measurements is ubiquitous in practical applications, leading to a growing demand for time series imputation to obtain higher-quality data. Unlike forecasting, which predicts future values based on historical observations, imputation focuses on reconstructing missing values using the available contextual information. Anomaly detection involves identifying unusual or abnormal patterns within a time series, which can indicate critical events, system faults, or outliers requiring further investigation. Lastly, classification assigns a label or category to a given time series based on its characteristics, a task widely utilized in fields such as medical diagnosis.

Given the pluralism of time series analysis, an array of deep learning models have emerged to address specific challenges in different tasks. Instead of a task-centric view into exhaustive details, this survey takes a model-centric view of deep time series models, focusing on their basic modules and architectural designs for addressing the foundational problems of capturing both temporal and inter-variable dependencies underlying large-scale time series datasets.

3 Basic Modules

In the past decade, time series modeling approaches have evolved significantly, shifting from traditional statistical models to sophisticated deep learning models. Despite these advancements, many classical tools and analytical algorithms remain widely used and continue to serve as foundational design principles in modern deep models. In this section, we focus on the major tools of classical time series analysis and demonstrate how they have been integrated as fundamental components in contemporary deep time series models.

3.1 Stationarization

Stationarity is a foundational concept in time series analysis, referring to the property where the statistical properties of a time series remain constant over time. A stationary time series exhibits a constant mean and variance, which simplifies statistical analysis and makes it easier to capture the underlying patterns and behaviors within a time series. Given that many statistics-based time series analysis methods fundamentally presuppose stationarity, the transformation of non-stationary data into a stationary form has become an essential preprocessing step. Conventionally, this stationarization is achieved through methods such as differencing or log-transformation. In recent deep learning approaches, data normalization [35] takes the role of stationarization in a simple but effective way, which standardizes the value distribution of observations while maintaining the intrinsic variations and further helps mitigate the distribution shift between the source and target domains.

Recent deep learning architectures have developed specialized layers to address the non-stationary problem. The deep adaptive input normalization (DAIN) layer [36] was proposed to adaptively standardize time series data based on its original distribution. RevIN [37] introduces a reversible instance normalization technique with learnable affine transforms to make the model bypass the non-stationary inputs. Further, Non-Stationary Transformer [38] (*Stationary* for short in the following) proposes a simpler but more effective series stationarization technique that improves the predictive capability of non-stationary series without extra parameters. Specifically, for a sequence with T time stamps and C variates $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_T\} \in \mathbb{R}^{T \times C}$, where $\mathbf{x}_t \in \mathbb{R}^C$, the outline of Stationary [38] can be summarized as follows:

$$\begin{aligned} \mu_{\mathbf{x}} &= \frac{1}{T}\sum_{i=1}^{T}\mathbf{x}_i,\ \sigma_{\mathbf{x}}^2 = \frac{1}{T}\sum_{i=1}^{T}(\mathbf{x}_i - \mu_{\mathbf{x}})^2, \\ \bar{\mathbf{X}} &= \frac{\mathbf{X} - \mu_{\mathbf{x}}}{\sqrt{\sigma_{\mathbf{x}}^2 + \epsilon}},\ \bar{\mathbf{Y}} = \mathrm{Model}(\bar{\mathbf{X}}),\ \hat{\mathbf{Y}} = \sigma_{\mathbf{x}}\bar{\mathbf{Y}} + \mu_{\mathbf{x}}, \end{aligned} \tag{1}$$

where ϵ is a small value for numerical stability. $\mu_{\mathbf{x}}, \sigma_{\mathbf{x}}^2 \in \mathbb{R}^{1 \times C}$ are the variate-specific mean and variance. To recover the distribution and non-stationarity of the original series, a denormalization module is further used to augment the model output $\bar{\mathbf{Y}}$ with mean and variance statistics of inputs.

The idea of stationarization and the aforementioned techniques have seen widespread adoption in subsequent deep time series models [23], [39], [40]. For instance, recent works such as SAN [41] rethink the nature of non-stationary data by normalizing non-overlapping, equally-sized slices instead of the entire series, and incorporating a statistics prediction module to forecast future slice distributions.

3.2 Decomposition

Decomposition [42], [43], as a conventional approach in time series analysis, breaks time series into several components of more regular patterns, with a divide-and-conquer paradigm to explore complex series variations. In the previous work, a set of typical decomposition schemes are explored.

3.2.1 Seasonal-Trend Decomposition

Seasonal-trend decomposition [44], [45] is one of the most common practices to enhance the predictability of raw time series data. This technique separates a series into distinct

additive components: trend, seasonal, cyclical, and irregular. The trend component represents the overall long-term pattern of the data over time, the cyclical component reflects repeated but non-periodic fluctuations within the data, the seasonal component indicates the repetitive patterns over a fixed period, and the irregular component is the residual or remainder of the time series after the other components have been removed.

Such decomposition can be achieved using mathematical tools like filters or exponential smoothing [46], [47]. Previous statistical approaches mainly adopt the trend-seasonality decomposition as data pre-processing [48]. In deep models, Autoformer [22] firstly introduces the idea of decomposition to deep learning architecture and proposes a *series decomposition block* as a basic module based on the average pooling to extract the seasonal and trend-cyclical parts of input series or deep features:

$$\begin{aligned} \mathbf{X}_{\mathcal{T}} &= \mathrm{AvgPool}\left(\mathrm{Padding}(\mathbf{X})\right), \\ \mathbf{X}_{\mathcal{S}} &= \mathbf{X} - \mathbf{X}_{\mathcal{T}}. \end{aligned} \tag{2}$$

Implemented based on a temporally average pooling layer, the proposed decomposition block effectively captures the trends $\mathbf{X}_{\mathcal{T}}$, and the remainder is taken as the seasonal part $\mathbf{X}_{\mathcal{S}}$. This design has been widely adopted as a native building block in the subsequent works [6], [49], [50], [51], [52] for disentangling underlying patterns within deep features.

3.2.2 Basis Expansion

Basis expansion is a mathematical method used to represent a function or a set of data points in terms of a new set of pre-defined functions. These new functions form the bases for a function space, allowing any function in that space to be expressed as a linear combination of these basis functions. In time series analysis, basis expansion is used to reveal non-linear temporal relationships by decomposing the time series into a combination of basic variations, thereby enhancing interpretability. As a representative, N-BEATS [24] presents hierarchical decomposition to time series by utilizing a fully connected layer to produce expansion coefficients for both backward and forward forecasts. For the l-th block in the proposed hierarchical architecture, the operation is

$$\begin{aligned} \mathbf{X}_l &= \mathbf{X}_{l-1} - \hat{\mathbf{X}}_{l-1} \\ \hat{\mathbf{X}}_l, \hat{\mathbf{Y}}_l &= \mathrm{Block}_l(\mathbf{X}_l), \end{aligned} \tag{3}$$

where $\hat{\mathbf{X}}_{l-1}$ is the backcast results which restrict the block to approximate the input signal $\mathbf{X}_{l-1}$, then $\mathbf{X}_l$ removes the portion of well-estimated signal $\hat{\mathbf{X}}_{l-1}$ from $\hat{\mathbf{X}}_{l-1}$, therefore providing a hierarchical decomposition. $\hat{\mathbf{Y}}_l$ is the partial forecast based on the decomposed input $\mathbf{X}_l$ and the final forecast $\hat{\mathbf{Y}} = \sum_l \hat{\mathbf{Y}}_l$ is the sum of all partial forecasts.

Subsequently, N-HiTs [53] redefines N-BEATS by incorporating subsampling layers before the fully connected blocks. This modification enhances input decomposition through multi-frequency data sampling and improves future prediction via multi-scale interpolation. DEPTS [54] proposes a novel decoupled formulation for periodic time series by introducing a periodic state as a hidden variable. It then develops a deep expansion module atop residual learning to perform layer-by-layer expansions between observed signals and these hidden periodic states. Similarly, DEWP [55] is a stack-by-stack expansion model designed to handle multivariate time series data. Each stack within the model comprises a variable expansion block for capturing dependencies among multiple variables and a time expansion block for learning temporal dependencies.

3.2.3 Matrix Factorization

The aforementioned two decomposition methods primarily address univariate series or are applied to multivariate series in a variate-independent way. Here, we explore factorization-based decomposition that is particularly pertinent to multivariate time series. These high-dimensional data can be formalized as a matrix where rows represent variates and columns denote time points. Since variables in multivariate time series tend to be highly correlated, they can be reduced to a more compact space. Matrix factorization methods [56] achieve this by decomposing the high-dimensional series data into the product of two matrices in a lower-dimensional latent space. For a multivariate time series $\mathbf{X} \in \mathbb{R}^{T \times C}$, the matrix can be approximated by the multiplications of two lower rank embedding matrix, $\mathbf{X} \approx \mathbf{F}\hat{\mathbf{X}}$, in which $\mathbf{F} \in \mathbb{R}^{T \times k}$, $\hat{\mathbf{X}} \in \mathbb{R}^{k \times C}$ and k is a hyperparameter.

Besides the estimation, regularizers are critical in factorization-based models to mitigate overfitting. Going beyond the canonical design that takes the squared Frobenius norm as regularizers, temporal regularized matrix factorization (TRMF) [57] designs an autoregressive-based temporal regularizer to describe temporal dependencies among latent temporal embeddings. Further, [58] extended TRMF with a new spatial autoregressive regularizer to estimate low-rank latent factors by simultaneously learning the spatial and temporal autocorrelations. For enhanced modeling of the trends and seasonality of real-world time series data, NoTMF [59] integrates the vector autoregressive process with differencing operations into the classical low-rank matrix factorization framework. Addressing the challenge of parameter tuning, BTF [60] offers a fully Bayesian approach, uniting probabilistic matrix factorization and the vector autoregressive process within a single probabilistic graphical model. Diverging from autoregressive-based temporal regularization, DeepGLO [61] leverages a temporal convolution network to capture non-linear dependencies. Similarly, LSTM-GL-ReMF [62] employs an LSTM-based temporal regularizer for complex long-term and short-term non-linear temporal correlations, complemented by a graph Laplacian spatial regularizer [63] for capturing spatial correlations.

3.3 Fourier Analysis

Fourier analysis [64], [65] is a powerful mathematical technique that transforms a physical signal into the frequency domain, thereby illuminating the inherent periodic properties of the original data. It has long been recognized as a fundamental analytical tool across diverse domains. Given that time series data are typically recorded as sequences of discrete observations sampled from continuous signals, Fourier analysis has naturally emerged as a mainstream approach in time series modeling with favorable effectiveness and efficiency [66], [67]. Introducing the Fourier domain not only enhances the representation of the original series but also provides a global perspective on temporal dynamics. The

resulting frequency spectrum distribution offers crucial insights into the essential periodic components and underlying cyclical behaviors of time series. In practice, algorithms such as Fast Fourier Transform (FFT) [68] and Wavelet Transform (WT) [69] act as foundational bridges, connecting the discrete temporal domain to the frequency domain. These techniques have gained increasing popularity in the modular design of deep time series models [70], [71], [72], [73], [74], [75], [76], [77]. Existing approaches can be broadly categorized into two main types: time-domain and frequency-domain modeling.

3.3.1 Time-Domain Modeling

The fundamental principle behind the Fourier transform is that sequential data can be decomposed and represented by a series of periodic signals. Consequently, it can be used to identify potentially dominant periods and their corresponding frequencies in the data by analyzing the highest amplitude components. As a prominent integration, TimesNet [3] employs the Fast Fourier Transform (FFT) to extract the most significant frequencies with the highest amplitude values. It then reshapes the 1D time series data into a 2D space based on these identified periods, thereby facilitating hierarchical representation learning. Building upon TimesNet, PDF [78] posits that frequencies with larger values facilitate a more discernible distinction between long-term and short-term relationships.

The frequency-domain representation provides valuable information regarding both amplitudes and phases. Low-frequency components, for instance, correspond to slower variations or underlying trends in the signal, while high-frequency components capture finer details or rapid fluctuations. A significant body of work has focused on leveraging frequency-domain information to enhance the model's capability in capturing temporal dependencies. FiLM [79] combines Fourier analysis with low-rank approximation and introduces a frequency enhanced layer (FEL). This approach retains portions of the representation related to low-frequency Fourier components and the top eigenspace, thereby effectively reducing noise and accelerating training speed. FITS [80] integrates a low-pass filter (LPF) to eliminate high-frequency components above a specified cutoff frequency, thereby compressing the model size while preserving essential information. Taking an opposing view, FEDformer [81] posits that retaining only low-frequency components is insufficient for time series modeling, as it risks dismissing important data fluctuations. Based on this consideration, FEDformer aims to capture a global view of time series by representing the series using a randomly selected constant number of Fourier components, encompassing both high-frequency and low-frequency contributions.

Beyond merely exploiting sequence information obtained from Fourier transforms, some research focuses on achieving efficient computation through the Fast Fourier Transform. Auto-correlation is a fundamental concept in time series analysis that measures the dependence between observations at different time points within a sequence of data. The Wiener-Khinchin theorem [82] provides a mathematical relationship between the auto-correlation function and the power spectral density (PSD) of a stationary random process, where the auto-correlation function represents the inverse Fourier transform of the PSD. Taking the data as a real discrete-time process, Autoformer [22] proposes an Auto-Correlation mechanism with an efficient Fast Fourier Transform to capture the series-wise correlation.

3.3.2 Frequency-Domain Modeling

Building on advancements in time-frequency analysis from signal processing, several approaches have been developed to analyze time series simultaneously in both the time and frequency domains. ATFN [83], for instance, utilizes an augmented sequence-to-sequence model to learn trending features of non-stationary time series, alongside a frequency-domain block for capturing periodic patterns. TFAD [84] introduces a time-frequency analysis-based model that employs temporal convolutional networks to obtain both time-domain and frequency-domain representations.

Some works have developed specialized deep learning architecture to process the frequency domain of time series. STFNet [71] applies Short-Time Fourier Transform to input signals and applies filtering, convolution, and pooling operations directly in the frequency domain. StemGNN [85] combines Graph Fourier Transform (GFT) and Discrete Fourier Transform to model both inter-series correlations and temporal dependencies. EV-FGN [86] uses a 2D discrete Fourier transform on the spatial-temporal plane of the embeddings and performs graph convolutions for capturing the spatial-temporal dependencies simultaneously in the frequency domain. FreTS [87] leverages Discrete Fourier Transform (DFT) to transform the data into the frequency domain spectrum and introduces frequency domain MLPs designed for complex numbers with separated modeling for the real parts and the imaginary parts. FCVAE [88] integrates both the global and local frequency features into the condition of Conditional Variational Autoencoder (CVAE) concurrently. Recent TSLANet [89] proposes a lightweight Adaptive Spectral Block (ASB) to replace the self-attention mechanism, which is achieved via Fourier-based multiplications by global and local filters. FourierDiffusion [90] extends the score-based SDE formulation of diffusion models to complex-valued data and therefore implements time series diffusion in the frequency domain.

4 Model Architectures

As discussed in Section 2, time series models aim to unearth the intrinsic temporal dependencies and variate correlations to uncover underlying patterns and dynamics within observations. Existing approaches can be broadly categorized into five groups based on their architectural backbones (Figure 2), namely, MLP-based, RNN-based, CNN-based, GNN-based, and Transformer-based. Each of these architectures possesses distinct strengths and design principles that address specific challenges in time series analysis. In this section, we provide a comprehensive technical review of current deep time series models, specifically highlighting their distinct approaches to time series modeling.

4.1 Multilayer Perceptrons

As a cornerstone of traditional statistical time series models, the Auto-regressive (AR) model posits that the model output depends linearly on its own past values. Inspired

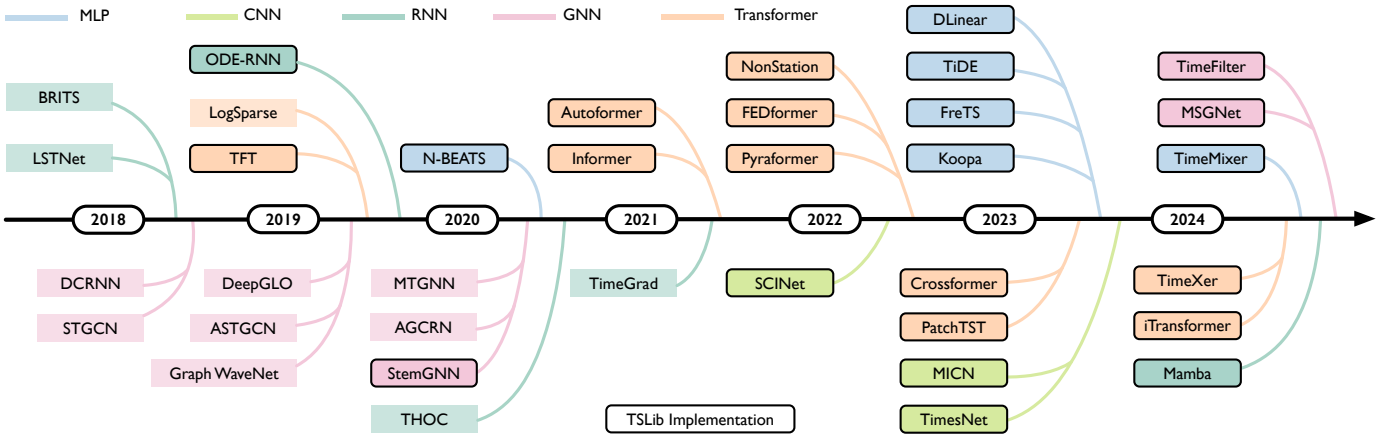


Fig. 2. An overview of representative time series models in chronological order. We mark models with different colors based on their architectures.

by the notable efficacy of auto-regressive models, Multilayer Perceptrons (MLP) have become a popular architecture for time series modeling.

As a prime example of MLP-based models, N-BEATS [24] integrates no time-series-specific components to capture temporal patterns. Specifically, as described in Eq. (3), N-BEATS consists of deep stacks of fully-connected layers with two residual branches in each layer, one for the backcast prediction and the other for the forecast branch. Expanding the idea of neural basis expansion, N-HiTs [53] enhances prediction through multi-rate signal sampling and hierarchical interpolation, while N-BEATSx [91] further extends the framework by incorporating exogenous variables.

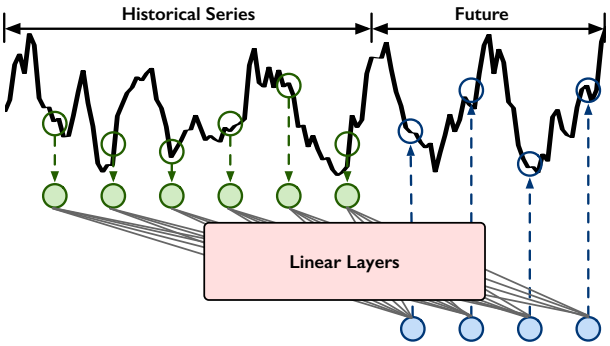


Fig. 3. Illustration of a basic MLP-based model in forecasting task, which captures the future-past dependencies with learnable MLP parameters.

Recent research, notably DLinear [49], rethinks the effectiveness of complicated deep architectures for temporal modeling. It argues a simple linear regression in the raw space that achieves remarkable performance in both modeling and efficiency. As illustrated in Figure 3, many prevalent MLP-based deep time series models, primarily designed for forecasting, rely on such simple linear layers. Also lightweight but effective, FITS [80] advocates that time series forecasting can be treated as interpolation exercises within the complex frequency domain. It introduces a complex-valued linear layer to learn amplitude scaling and phase shift directly in the frequency domain. Inspired by MLP-Mixer [92] in computer vision, several works have attempted to leverage MLPs to model both temporal and variate dependencies. TSMixer [93] contains interleaving time-mixing and feature-mixing MLPs to extract information from different perspectives. To better capture global dependencies, FreTS [87] investigates the learned patterns of frequency-domain MLPs, which operate on both inter-series and intra-series scales to capture channel-wise and time-wise dependencies in multivariate data.

Moving beyond simple linear layers over discrete time points, TimeMixer suggests that time series exhibit distinct patterns across different sampling scales and introduces an MLP-based multiscale mixing architecture. TiDE [94] incorporates exogenous variables to enhance time series prediction. Based on Koopman theory and Dynamic Mode Decomposition (DMD) [95], which is a dominant approach for analyzing complicated dynamical systems, Koopa [96] hierarchically disentangles dynamics through an end-to-end predictive training framework and can utilize real-time incoming series for online development.

4.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are specifically designed for sequential data [97], [98], [99], such as natural language processing [100] and audio modeling [101]. Given the inherent sequential nature of time series, RNNs have naturally become a popular choice for time series analysis [102]. Existing RNN-based deep time series models primarily focus on combating the gradient vanishing problem caused by the vanilla recurrent structure and modeling the mutual correlation among multivariate variables.

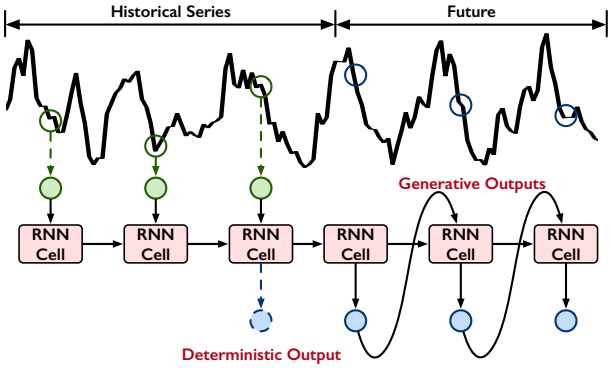


Fig. 4. Illustration of RNN-based model in the forecasting task.

Numerous prior works [103], [104], [105], [106], [107] have leveraged various RNN variants to capture complex

dependencies. LSTNet [108] combines the recurrent structure with the convolutional layer to capture both the short-term local dependency between variables and long-term patterns for time series. It further introduces a novel recurrent-skip component, based on periodic patterns, to alleviate gradient vanishing in modeling long-term dependencies. Similarly, DA-RNN [109] combines the recurrent unit with a dual-stage attention mechanism to adaptively extract relevant series at each time step. Beyond deterministic forecasts, DeepAR [110] proposes an auto-regressive recurrent network to predict the probability distribution of further time points. Technologically, it learns not only the seasonal behavior with time series but dependencies on given covariates, allowing the model to make predictions even when there is little or no historical data. Recent work [75] introduces a novel DTWAtt module, which integrates a data-dependent wavelet function into the BiLSTM network as wavelet attention to analyze dynamic frequency components in nonstationary time series.

Based on Markovian state representation, the State Space Model (SSM) [111] is a classical mathematical framework for capturing probabilistic dependencies between observed measurements in stochastic dynamical systems. While SSMs have proven their effectiveness and efficiency in processing well-structured time series data, traditional approaches have to refit each series separately and therefore cannot infer shared patterns from a dataset of similar time series. With the advent of deep learning, modern SSMs are often implemented in a recurrent manner. By adapting and propagating a deterministic hidden state, RNNs offer an alternative to classical state space models by representing long-term dependencies in continuous data. Consequently, some works [112], [113] have sought to fuse classical state space models with deep neural networks. Representative among these, the Deep State Space Model (DSSM) [114] leverages a recurrent neural network (RNN) to parametrize a specific linear SSM, thereby incorporating structural assumptions while learning complex patterns. The Structured State Space sequence model (S4) [115] introduces a novel parametrization for the SSM by conditioning matrix $\mathbf{A}$ with a low-rank correction, which allows for stable diagonalization and significantly enhances the model's long-term modeling capacity. More recently, Mamba [116] has emerged as a powerful method for modeling long sequential data with linear complexity w.r.t. sequence length. Utilizing a simple selection mechanism that parameterizes the SSM parameters based on input, Mamba can discern valuable information in observations, posing an effective way for time series analysis.

Going beyond learning among discretized time steps, Neural Ordinary Differential Equations (Neural ODEs) [117] extend RNNs into a continuous-time version. Technically, Neural ODEs parameterize the derivative of hidden states through neural networks, enabling seamless modeling of temporal evolution in a differentiable manner. Given the ability to model continuous-time, several works have explored applying Neural ODEs to handle irregularly-sampled time series data [118], [119]. Building on these advancements, [120] integrates Neural ODEs with physical priors to augment physical models with deep networks for forecasting complex dynamics. These capabilities make Neural ODEs a compelling complement to RNN-based methods, offering a more flexible and robust approach for time series analysis.

4.3 Convolutional Neural Networks

Since the semantic information of time series is mainly hidden in the temporal variation, Convolutional neural networks (CNN) [18], [121] have become a competitive backbone. Their strength lies in their ability to capture local features and recognize intricate patterns, which has been demonstrated with remarkable success in various computer vision tasks, including image classification [122], segmentation [123], and object detection [124].

Considering the temporal continuity of time series data, previous works [125], [126] apply a one-dimensional CNN (1D CNN) to discern the local patterns of time series data. Recent SCINet [127] employs canonical convolutions with a hierarchical downsample-convolve-interact architecture to capture dynamic temporal dependencies across varying temporal resolutions. Inspired by the idea of masked convolution [25], Wavenet [128] introduces causal convolution and dilated causal convolution to model long-range temporal causality. Similarly, Temporal Convolutional Networks (TCN) [129] uses a stack of dilated convolutional kernels with progressively enlarged dilation factors to achieve a large receptive field. However, the limited receptive field of TCN often hinders its capacity to capture global relationships in time series data. Based on TCN, MICN [130] is a local-global convolution network that combines different convolution kernels to model temporal correlation from a local and global perspective. ModernTCN [131] enhances the traditional TCN by separately employing Depthwise Separable Convolutions (DWConv) and Convolutional Feed-Forward Networks (ConvFFN) to capture cross-time and cross-variable dependencies, respectively.

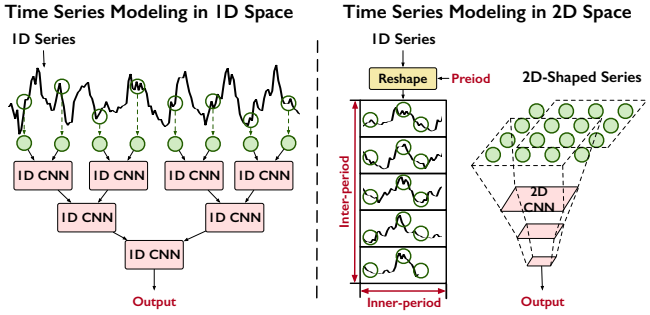


Fig. 5. Comparison of existing CNN-based time series models from the perspective of representation space.

Motivated by the inherent periodicity of time series data, TimesNet [3] goes beyond traditional 1D modeling. It [3] transforms the 1D time series $\mathbf{X}_{1\mathrm{D}}$ data into a set of 2D tensors $\mathbf{X}_{2\mathrm{D}} = \{\mathbf{X}^1_{2\mathrm{D}}, \cdots, \mathbf{X}^k_{2\mathrm{D}}\}$ in each TimesBlock based on the estimated period lengths, where the inter-period variations are presented in tensor columns and inner-period ones are shown in tensor rows. Here k is a hyperparameter, corresponding to multiple 1D-to-2D transformations with different periods. Subsequently, TimesNet applies an Inception Block [132], [133] to process these transformed 2D tensors, which can be summarized as:

$$\begin{aligned}
\mathbf{X}^i_{2\mathrm{D}} &= \mathrm{Reshape}\left(\mathrm{Padding}(\mathbf{X}_{1\mathrm{D}})\right),\ i \in \{1, \cdots, k\} \\
\hat{\mathbf{X}}^i_{2\mathrm{D}} &= \mathrm{Inception}\left(\mathbf{X}^i_{2\mathrm{D}}\right),\ i \in \{1, \cdots, k\} \\
\hat{\mathbf{X}}^i_{1\mathrm{D}} &= \mathrm{Trunc}\left(\mathrm{Reshape}\left(\hat{\mathbf{X}}^i_{2\mathrm{D}}\right)\right),\ i \in \{1, \cdots, k\},
\end{aligned} \tag{4}$$

where $\mathbf{X}_{2D}^{i}$ is the i-th transformed 2D tensor. After passing through the inception block $\mathrm{Inception}(\cdot)$, the learned 2D representations are transformed back to 1D for aggregation. These transformations enable TimesNet to effectively capture both multi-scale intraperiod-variation and interperiod-variation simultaneously. Furthermore, by leveraging hierarchical convolutional layers, TimesNet is capable of learning both high-level and low-level representations, facilitating comprehensive time series analysis across four distinct tasks.

4.4 Graph Neural Networks

Analyzing multivariate time series data is often challenging due to the complicated and often nonlinear correlations between variables. To address this, Graph neural networks (GNNs) [134], [135] have been widely adopted in time series analysis. As depicted in Figure 6, by modeling multivariate data as a spatiotemporal graph, where each node represents a variable, GNNs can extract relationships among neighboring nodes and capture the temporal evolution of node attributes over time, providing a framework for understanding the underlying dynamics of multivariate time series.

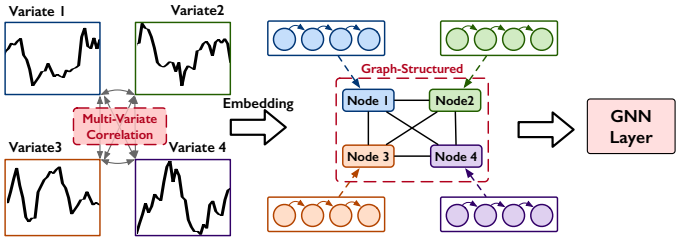


Fig. 6. Illustration of modeling multivariate time series using GNN.

The primary goal of GNN architecture is to model the latent topological relations in multivariate data. Accordingly, existing GNN-based works can be broadly categorized into two types: those that rely on a pre-defined graph structure as input and those that learn the graph structure dynamically. DCRNN [136] models the spatial dependency of traffic as a diffusion process on a directed graph and uses diffusion convolution to capture the spatial dependency, alongside a recurrent neural network to capture temporal dynamics. Similarly, STGCN [137] integrates graph convolutional networks to model the spatial dependencies among traffic sensors with temporal convolutions to capture the temporal dependencies in the traffic time series data. Graph WaveNet [138] combines graph convolution with dilated causal convolution and learns an adaptive dependency matrix through node embedding, enabling the model to automatically capture hidden spatial dependencies in spatial-temporal graph data. Similarly, AGCRN [139] enhances the traditional graph convolutional network with node adaptive parameter learning and data-adaptive graph generation modules, allowing for the automatic capture of spatial and temporal correlations without a pre-defined graph structure. MTGNN [140] introduces a graph learning layer to adaptively learn the graph adjacency matrix, thereby capturing hidden relationships among multivariate time series data. STFGNN [141] employs a Spatial-Temporal Fusion Graph Neural Network with a generated temporal graph to learn localized spatial-temporal heterogeneity and global spatial-temporal homogeneity. StemGNN [85] leverages the advantages of both the Graph Fourier Transform (GFT) and the Discrete Fourier Transform (DFT), modeling multivariate time series in the spectral domain.

4.5 Transformers

In view of the great success in the field of natural language processing [16], [142], [143], [144], [145] and computer vision [19], [146], [147], [148], Transformers have emerged as a powerful backbone for time series analysis. Benefiting from the self-attention mechanism [149], Transformer-based models excel at capturing long-term temporal dependencies and entangled multivariate correlations. As overviewed in Figure 7, existing Transformer-based time series models can be categorized based on the granularity of representation used in the attention mechanism, namely point-wise, patch-wise, and series-wise approaches.

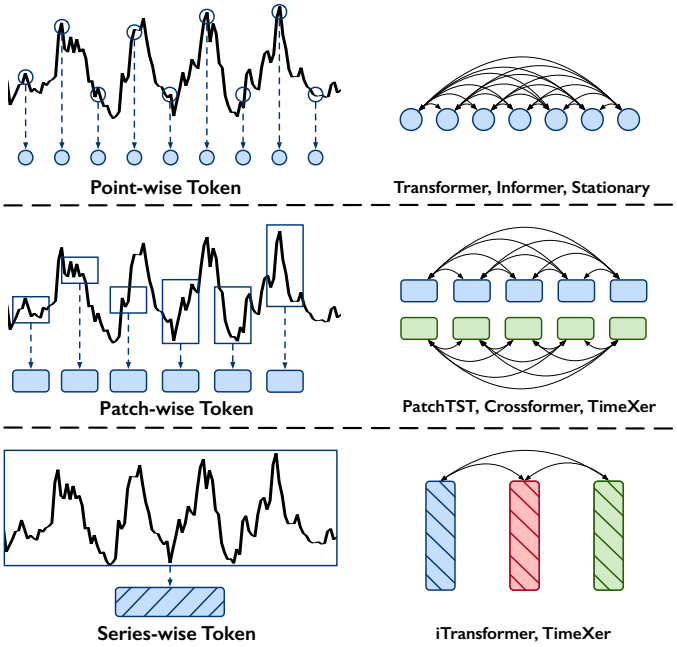


Fig. 7. Comparison of different types of tokenization used for Transformer-based time series models and corresponding representative work, ranging from point-wise, patch-wise to series-wise.

4.5.1 *Point-wise Dependency*

Due to the serial nature of time series, most existing Transformer-based works learn point-wise representations and apply attention mechanisms to capture the correlations among different time points. Among these point-wise modeling approaches, Data Embedding stands as a crucial component, mapping raw time series values into a high-dimensional representation. Given time series $\mathbf{X} \in \mathbb{R}^{T\times C}$ with corresponding time stamp information $\mathbf{X}^{\mathrm{ts}} \in \mathbb{R}^{T\times D}$, where C is the variate number and D is the types of time stamps, the embedding module can be summarized as:

$$\mathbf{H}_t = \mathrm{Projection}(\mathbf{X}_t) + \mathrm{PE}(\mathbf{X}_t) + \mathrm{TE}(\mathbf{X}_t^{\mathrm{ts}})), \tag{5}$$

where $\mathbf{H}_t \in \mathbb{R}^{T\times d_{\mathrm{model}}}$ and d_{model} is the dimension of the emebedded representation, value projection $\mathrm{Projection} : \mathbb{R}^{C} \mapsto \mathbb{R}^{d_{\mathrm{model}}}$ and timestamp embedding $\mathrm{TE} : \mathbb{R}^{D} \mapsto$

TABLE 2
Model cards of Transformer-based deep time series models with architectural details.

Category	Method	Architecture	Embeddding	Attention Mechanism
Vanilla	Transformer	Enc-Dec	Standard	$\text{FullAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V}$
Point-wise	LogSparse [150]	Dec-only	Standard	$\widehat{\mathbf{Q}},\widehat{\mathbf{K}} = \text{CausalCov}(\mathbf{H})$ $\text{FullAttention}(\widehat{\mathbf{Q}},\widehat{\mathbf{K}},\mathbf{V}) = \text{Softmax}(\frac{\widehat{\mathbf{Q}}\widehat{\mathbf{K}}^\intercal}{\sqrt{d}})\mathbf{V}$
	Informer [21]	Enc-Dec	Standard	$\overline{\mathbf{Q}} = \text{Top}_u\left(\overline{M}(\mathbf{q}_i,\mathbf{K})\right)$ $\text{ProbSparse-Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\overline{\mathbf{Q}}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V}$
	Pyraformer [151]	Enc-Dec	Standard	$\text{Pyramid-Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Masked}(\text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V})$
Patch-wise	Autoformer [22]	Enc-Dec	Standard	$\text{Auto-Correlation}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \sum_{i=1}^{k}\text{Roll}(\mathbf{V},\tau_i)\widehat{\mathcal{R}}_{\mathbf{Q},\mathbf{K}}(\tau_i)$
	Crossformer [152]	Enc-Dec	Patch-Wise	$\text{FullAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V}$
	PatchTST [23]	Enc-only	Patch-Wise	$\text{FullAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V}$
Variate-wise	iTransformer [39]	Enc-only	Variate-Wise	$\text{FullAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}})\mathbf{V}$

$\mathbb{R}^{d_{\text{model}}}$ are implemented by channel-dimension linear layers, and $\text{PE}(\cdot)$ denotes the absolute position embedding to preserve the sequential context of input series.

To optimize the application of Transformer architecture to the time series domain, existing literature primarily falls into two categories: designing pre-processing modules and modifying the attention mechanisms. As previously discussed in Section 3.1, techniques like RevIN [37] and Stationary [38] have achieved superior performance by integrating Normalization and De-Normalization modules before and after the Transformer blocks. Notably, Stationary [38] further introduces De-stationary Attention to mitigate the problem of over-stationarization.

Given that canonical attention leads to a quadratic computational complexity, various efficient attentions [21], [22], [81], [151] have been proposed to mitigate the complexity arising from point-wise modeling, which is summarized in Table 2. LogSparse [150] proposes *Convolutional Self-Attention* to replace canonical attention by employing causal convolutions to produce queries and keys in the self-attention layer. Informer [21] introduces a *Query Sparsity Measurement*, where a larger value indicates a higher chance of containing the dominant information in self-attention. Building on this, it designs ProbSparse self-attention, which selectively uses only the top queries with the highest measurement results, thereby significantly reducing quadratic complexity in both time and memory. Pyraformer [151] constructs a multi-resolution tree and develops a *Pyramidal Attention Mechanism*, in which every node can only attend to its neighboring, adjacent, and children nodes. With a calculated mask for attention, Pyraformer captures both short- and long-temporal dependencies with linear computational complexity.

4.5.2 Patch-wise Dependency

Patch-based architectures play a crucial role in the Transformer models for both Natural Language Processing (NLP) [16] and Computer Vision (CV) [19]. Since point-wise representations often fall short in capturing local semantic information in temporal data, several studies [23], [153], [154] have been devoted to exploring patch-level temporal dependencies in time series data.

Pioneer work Autoformer [22] proposes an *Auto-Correlation Mechanism* to replace canonical point-wise self-attention, enabling the model to learn series-wise dependencies. Based on the stochastic process theory [155], Auto-Correlation utilizes the Fast Fourier Transform to discover the time-delay similarities between different sub-series. A time delay module is further proposed to aggregate similar sub-series from underlying periods instead of the relation between scattered points, which firstly explores the sub-series level modeling in Transformer-based models.

Different from modifying the attention mechanism, most recent works utilize patch-wise representations of the data and perform a canonical self-attention mechanism to capture patch-wise dependencies [23], [51], [156]. PatchTST [23] and its subsequent works split time series $\mathbf{X}$ into a sequence of overlapped patches and embed each patch following:

$$\begin{aligned}\{\mathbf{P}_1,\mathbf{P}_2,\cdots,\mathbf{P}_N\} &= \text{Patchify}\left(\mathbf{X}\right),\\ \mathbf{H}_i &= \text{PatchEmbed}(\mathbf{P}_i) + \mathbf{E}^i_{\text{pos}}.\end{aligned} \tag{6}$$

Let P denote the patch length and N the corresponding number of patches, and $\mathbf{P}_i$ is the i-th patch of length P. The patches are mapped to the latent space through a temporal linear projection $\text{PatchEmbed} : \mathbb{R}^P \mapsto \mathbb{R}^{d_{\text{model}}}$ and a learnable position embedding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{d_{\text{model}}\times N}$. Based on the vanilla attention mechanism, PatchTST [23] learns the patch-wise dependencies. Going beyond PatchTST, Pathformer [157] introduces a multi-scale Transformer that employs adaptive pathways to dynamically select patch sizes, thereby learning multi-scale temporal representations.

The success of PatchTST also benefits from its channel-independence design, where each temporal patch-level token encapsulates information from only a single series. In addition to capturing the patch-level temporal dependencies within one single series, recent approaches [51], [156] have endeavored to capture interdependencies among patches from different variables over time. Crossformer [152] introduces a *Two-Stage Attention layer* containing a Cross-Time Stage and a Cross-Dimension Stage to efficiently capture the cross-time and cross-variate dependencies between each patch token. For the obtained embedded vector $\mathbf{H} \in \mathbb{R}^{N\times C\times d_{\text{model}}}$, the overall attention stage can be described as follow:

$$\begin{aligned}\mathbf{Z}^{\text{time}} &= \text{Attention}^{\text{time}}\left(\mathbf{H},\mathbf{H},\mathbf{H}\right)\\ \mathbf{B} &= \text{Attention}^{\text{dim}}_1\left(\mathbf{R},\mathbf{Z}^{\text{time}},\mathbf{Z}^{\text{time}}\right)\\ \overline{\mathbf{Z}}^{\text{dim}} &= \text{Attention}^{\text{dim}}_2\left(\mathbf{Z}^{\text{time}},\mathbf{B},\mathbf{B}\right),\end{aligned} \tag{7}$$

where $\mathbf{R} \in \mathbb{R}^{\mathbf{N}\times\mathbf{C}\times d_{\text{model}}}$ is a learnable vector array used as a router to gather information from all dimensions and then distribute the gathered information.

4.5.3 Series-wise Dependency

Further expanding the receptive field, recent works tokenize the entire time series to capture inter-series dependencies. iTransformer [39] introduces VariateEmbed for multivariate data and for i-th variable $\mathbf{X}^{(i)}$, it can be formulated as:

$$\mathbf{H}^{(i)} = \text{VariateEmbed}(\mathbf{X}^{(i)}) \tag{8}$$

where $\text{VariateEmbed} : \mathbb{R}^T \to \mathbb{R}^{d_{\text{model}}}$ is instantiated as trainable linear projector. Based on the global representations of each series, iTransformer utilizes the vanilla Transformer without any architectural modifications to capture mutual correlations in multivariate time series data. Similarly, TimeXer [158] focuses on forecasting with exogenous variables and utilizes patch-level and series-level representations for endogenous and exogenous variables, respectively. Additionally, an endogenous global token is introduced to TimeXer, which serves as a bridge in-between and therefore captures intra-endogenous temporal dependencies and exogenous-to-endogenous correlations jointly.

5 Time Series Foundation Models

With the advent of Large Language Models (LLMs), the large-scale pre-training paradigm has inspired our community moving towards Time Series Foundation Models (TSFMs) to learn complex temporal dependencies and capture the underlying dynamics inherent in large-scale time series data. ForecastPFN [159] introduces the first zero-shot forecasting model trained exclusively on a novel synthetic data distribution. Recently, numerous large-scale time series models have been proposed, primarily leveraging Transformer-based architectures. These models are pre-trained on massive time series datasets, often at the billion- or trillion-scale, to achieve strong zero-shot generalization for downstream tasks. Technologically, these models can be broadly categorized into two types: time series native foundation models built from scratch and adapted from LLMs.

5.1 Time-Series Native TSFM

Encoder-based TSFM Inspired by the success of encoder-only deep time series forecasters, some encoder-based TSFMs have been proposed. MOMENT [160] is a Transformer encoder pre-trained with a mask reconstruction objective on univariate time series data, which can be fine-tuned for various downstream analysis tasks. Moirai [161] introduces a multivariate pre-training paradigm by flattening variates and considering all variates as a single sequence. The successor Moirai-MoE [162] utilizes sparse experts to model diverse time series patterns in a data-driven manner, enabling automatic token-level specialization. Additionally, researchers have explored TSFMs within alternative architectural frameworks. TTMs [163] employs a lightweight architecture consisting of Multi-Layer Perceptrons and integrates techniques such as adaptive patching, diverse resolution sampling, and resolution prefix tuning to facilitate pre-training on datasets with varied resolutions while maintaining minimal

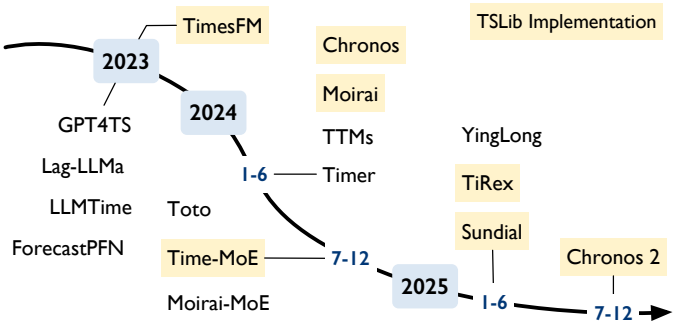


Fig. 8. Recent development of time series foundation models. The timeline is established according to the release date of the paper or technical report for the model.

model capacity. TiRex [164], based on xLSTM, incorporates a novel masking strategy that retains state-tracking capabilities, allowing pre-trained time series models to provide reliable uncertainty estimates over extended prediction horizons.

Decoder-based TSFM Motivated by the substantial progress of decoder-only LLMs, another line of research adopts decoder-only architectures for rollout generation. Lag-Llama [165] is a decoder-only Transformer that uses time series lags as covariates. TimesFM [166] is a decoder-style attention model with input patching, using a large time-series corpus comprising both real-world and synthetic datasets. Going beyond forecasting tasks, Timer [154] converts forecasting, imputation, and anomaly detection of time series into a unified generative task and proposes a generative pre-trained Transformer for general time series analysis. Time-MoE [167] introduces a scalable, unified architecture with a sparse mixture-of-experts design, extending the training corpus to 300 billion time points. Previous works rely on the inherent generative ability of the pre-trained Transformer architecture. Recently, Sundial [168] proposes TimeFlow based on flow-matching generation to predict next-patch's distribution, allowing Transformers to be trained without discrete tokenization and make probable predictions. Pretrained on one trillion time points, Sundial demonstrates superior performance on both point and probabilistic forecasting with a fast inference speed. Tailored for high-dimensional observability, Toto [169] introduces an advanced attention mechanism that allows for efficient grouping of multivariate time series features, reducing computational overhead while maintaining high accuracy.

5.2 LLM Empowered TSFM

Based on the similar sequential nature, fine-tuning pre-trained language models to equip them with time series analysis capabilities has become a promising research topic. When applying LLMs to time series, it is essential to tokenize the time series before feeding it to a pre-trained model.

GPT4TS [153] proposes a unified framework for diverse time series analysis tasks by using a pre-trained GPT-2 model and fine-tuning the positional embeddings and the parameters of the layer normalization for each analysis task. LLMTime [170] introduces a tokenization scheme that encodes real-valued data as a string of digits after fixing the numerical precision and scaling the data appropriately. Based on the T5 family, Chronos [171] introduces a pre-trained

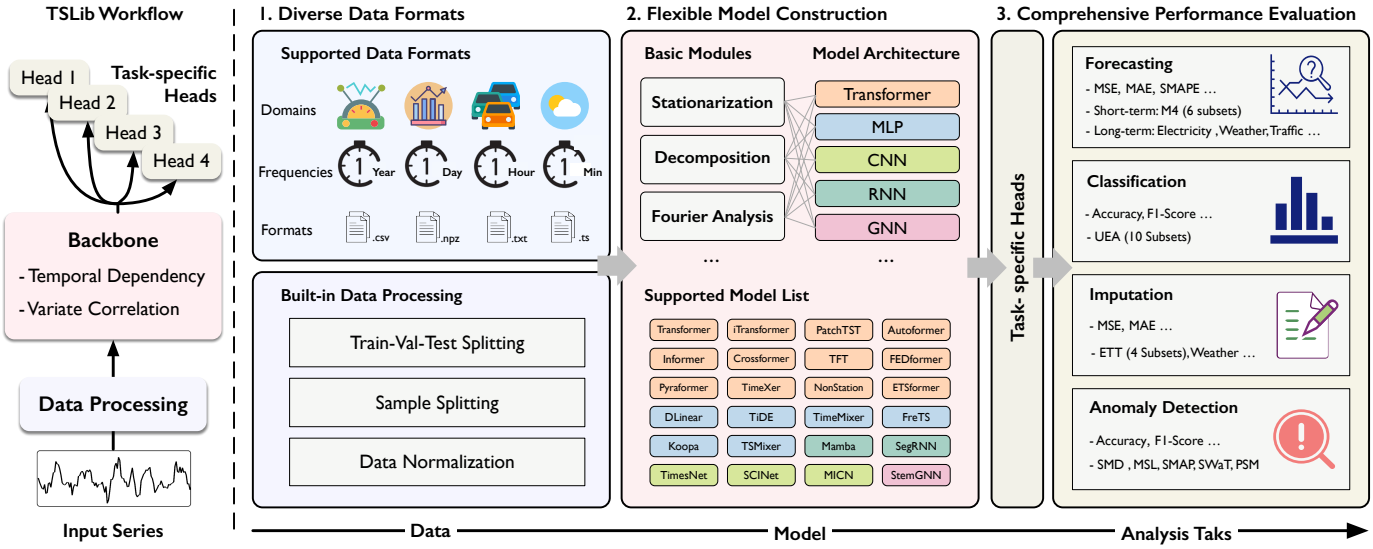


Fig. 9. Overall framework of Time Series Library (TSLib). Left: Workflow for task-general experiments. Right: Key features and design principles.

probabilistic time series forecaster on a large collection of publicly available datasets, complemented by a synthetic dataset. Technologically, Chronos tokenizes time series values using scaling and quantization into a fixed vocabulary and trains the model on these tokenized time series via the cross-entropy loss. Recently, Chronos2 [172] extends univariate forecasting to universal forecasting by incorporating a novel group attention mechanism.

5.3 Discussion

The selection between time series native foundation models and LLM empowered TSFMs hinges on the trade-off between specialized temporal modeling and generalized reasoning. Native TSFMs are fundamentally designed to capture fine-grained temporal patterns, offering superior fidelity to intricate variations via architectures tailored for numerical data. Consequently, they serve as the preferred paradigm for purely numerical scenarios, providing stable and efficient modeling of intrinsic dynamics to deliver rigorous point estimations. LLM empowered TSFMs, conversely, excel in cross-modal synthesis by leveraging inherent semantic priors and reasoning abilities of LLMs. These models are particularly advantageous for complex scenarios where contextual metadata, such as textual descriptions or event logs, can be aligned with temporal patterns or where internal world knowledge is required to facilitate modeling. Therefore, while native models define the frontier of dedicated numerical prediction, LLM empowered paradigms offer a versatile alternative for context-aware and cross-domain applications.

6 Time Series Library

Recently, extensive exploration of deep time series methods has resulted in significant advances. Given such booming development, the issue of fair benchmarking poses a pressing challenge for future research. Thus, focusing on time series analysis, several benchmarks have been proposed, such as DGCRN [173], LibCity [174], DL-Traff [175], TS-bench [176], and BasicTS [14]. Besides, Autoformer [22] proposed a standard long-term forecasting benchmark covering different practical applications. Further, to verify the generality of different time series analysis models, TimesNet [3] builds a more comprehensive model generalization benchmark covering five mainstream time series analysis tasks. However, these benchmarks typically have some limitations. One issue with current time series benchmarks is their limited coverage of time series analysis tasks and specific domains, which limits their practical applications. Moreover, these benchmarks often fail to provide detailed discussions and comprehensive summaries of task types, model architectures, and specific baseline methods. As a result, they do not effectively guide the design of more efficient time series methods or drive further development in the field.

To address these challenges effectively, we introduce Time Series Library (TSLib), a benchmark designed for the fair and comprehensive evaluation of deep time series models across a wide range of analysis tasks. As shown in Figure 9, TSLib provides a unified experimental framework that includes standardized evaluation protocols, a rich and diverse collection of real-world datasets, state-of-the-art and widely-used time series models, and a streamlined process for experimental validation and analysis. Although numerous time series libraries have been proposed in recent years, TSLib stands out by including advanced models and supporting a wide range of time-series analysis tasks, which have earned widespread recognition from the community as shown in Table 3. Specifically, we meticulously followed the official codes and implemented 41 widely used and advanced deep time series models. Users can choose from these models based on their specific practical usage scenarios. The code is available at https://github.com/thuml/Time-Series-Library.

6.1 Design and Implementation Principles

TSLib is designed based on the well-established factory pattern and implements a unified interface between data, model, and experiment objects, thus enabling a clear separation between deep model creation and usage, promoting modularity and flexibility. By loading different data and model objects and combining task-specific heads, TSLib

TABLE 3
Comparison between TSLib and other related libraries. * denotes the absence of deep models. "Stars" refers to GitHub stars as of Feb. 2026.

Library	#Models	Task	Stars
tslearn* [177]	10	Clustering, Classification, Regression	3.1k
StatsForecast* [178]	31	Forecasting, Anomaly Detection	4.7k
NeuralForecast [179]	33	Forecasting	4.0k
GluonTS [180]	25	Forecasting, Anomaly Detection	5.1k
TSLib (This Survey)	41	Forecasting, Imputation Classification, Anomaly Detection	11.6k

enables different datasets and models to be shared and extended, allowing easy switching between various datasets, models, and analysis tasks. Here are the key features.

Diverse Data Formats TSLib natively supports a wide range of datasets in a variety of formats, including ".csv", ".npz", ".txt", etc. As summarized in Figure 9, TSLib currently supports more than 30 datasets with different sampled frequencies across four mainstream time series analysis tasks, all derived from real-world scenarios, such as energy, transportation, economics, weather, and medicine etc. Moreover, TSLib excels in scalability, allowing for the effortless integration of new data sources of different types. Besides, to support subsequent training and evaluation, TSLib includes a multitude of data processing steps, including time window splitting, data batch generation, etc, where the raw data is partitioned into separate windows for training, validation, and testing purposes, enabling streamlined model training and equitable comparisons. These steps serve as indispensable prerequisites for attaining precise and dependable results across a range of diverse time series analysis tasks.

Moreover, TSLib provides additional support for numerous crucial and effective data processing strategies [36], [37], [38], [42] to enhance model performance and training efficiency. We encapsulate these various design strategies as basic data processing layers, encompassing techniques such as data normalization, time-frequency decomposition, Fourier analysis, and more. When utilizing TSLib, users have the flexibility to select these strategies to improve the training effect based on their specific requirements and objectives.

Flexible Model Construction As stated in the previous sections, we abstract the construction of deep time series models as the combination of basic modules (Section 2) and model architectures (Section 4). Specifically, we separate the most commonly used modules, such as stationarization and decomposition, from previous elaborately designed models. In this way, it is very convenient to reproduce or add a deep model with pre-defined basic modules. Further, although different models are originally proposed for different tasks, such as Autoformer [22] is for forecasting and TimesNet [3] is for more generic tasks, following the "model-centric" principle in this survey, we unify diverse analysis tasks as a representation learning task with domain-specific heads, enabling a task-general evaluation for different backbones.

Comprehensive Performance Evaluation Building upon the above two essential features, TSLib can naturally support the evaluation of diverse models on different tasks based on extensive datasets, building a comprehensive benchmark. Specifically, in TSLib, we provide evaluation for four mainstream time series analysis tasks: classification, imputation, forecasting, and anomaly detection. Each task involves specific evaluation metrics, enabling a comprehensive assessment of the performance of models. Noticing that, we further distinguish the forecasting task as long- and short-term tasks following the previous convention, where the separation depends on the forecasting horizon.

6.2 Evaluation Protocols and Metrics

To conduct a fair and comprehensive model comparison, TSLib provides standardized evaluation protocols for four mainstream time series analysis tasks following [3]. Specifically, different models are trained and tested based on the same training strategies and test sets to avoid potential unaligned configurations and ensure a rigorous comparison among different models.

Following previous research, for long-term forecasting and imputations, we adopt Mean Square Error (MSE) and Mean Absolute Error (MAE) as the primary evaluation metrics. For short-term forecasting, we use the Symmetric Mean Absolute Percentage Error (SMAPE) and Mean Absolute Scaled Error (MASE) as metrics, which focus on absolute errors and reduce the impact of outliers, providing reliable evaluations of forecast accuracy across different datasets and methodologies. In time series classification, we utilize classification accuracy as the evaluation metric. For anomaly detection, we employ the F1 score to validate the identification of abnormal values. The F1 score represents a balanced combination of precision and recall, offering a comprehensive assessment of a classifier's performance, especially when dealing with imbalanced abnormal classes.

6.3 Supported Datasets

TSLib natively supports a variety of widely used datasets across different numbers of samples and variables, covering diverse domains and tasks. The correspondences between tasks and datasets are depicted in Figure 9. Since all datasets are commonly used, here is only a brief introduction, and more details can be found in previous papers [3].

Forecasting Forecasting is one of the most widely explored tasks in previous research, bringing us a wide choice of experimental datasets. Just as stated in design principles, we further divide this task into two types (long- and short-term) according to the forecasting horizon. For long-term time series forecasting, TSLib includes nine datasets, including Electricity Transformer Temperature (ETT, including four subsets) [21], Electricity, Weather, Traffic, Exchange, and Illness [22]. For the short-term forecasting task, we include the M4 dataset [181], which comprises six sub-datasets with varying sampling frequencies and domains.

Imputation Due to glitches, the collected time series data may contain partially missing values, posing a challenge for time series analysis. However, in real-world applications, it is really hard to obtain the ground truth values for the missing records. Thus, we utilize the Electricity Transformer Temperature (ETT, with four subsets) [21], Electricity, and Weather to construct the missing value situation by randomly masking a certain portion of time steps.

Anomaly Detection Anomaly detection involves identifying unusual or abnormal patterns in a time series. These anomalies can indicate critical events, faults, or outliers that require attention or further investigation. There are some

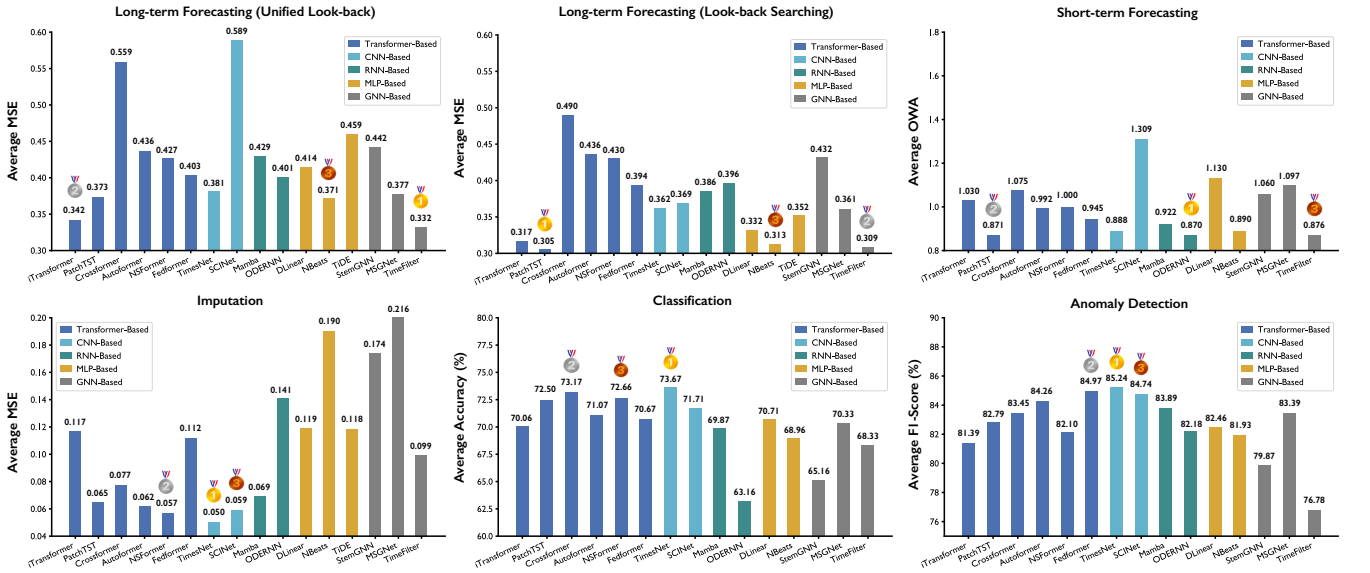


Fig. 10. Comparison of model performance based on the Time Series Library. Full results are averaged from a diverse set of datasets supported by TSLib across four mainstream analysis tasks. In the classification and anomaly detection tasks, higher Accuracy or F1-score indicates better model performance. Conversely, lower average MSE or MAE demonstrates superior model effectiveness for the other tasks. The top three models for each time analysis task are highlighted on a leaderboard, and the optimal model for each time series analysis task is indicated.

mainstream anomaly detection datasets supported in TSLib, such as Server Machine Dataset (SMD) [182], Mars Science Laboratory rover (MSL) [183], Soil Moisture Active Passive satellite (SMAP) [183], Secure Water Treatment (SWaT) [184], and Pooled Server Metrics (PSM) [185] which are collected from a variety of industrial scenarios.

Classification We selected ten multivariate datasets from the UEA Time Series Classification Archive [186] to test the series-level classification tasks. These datasets cover a range of practical tasks, including gesture, action, audio recognition, and medical diagnosis through heartbeat monitoring. We pre-processed the datasets according to the descriptions in [187].

6.4 Default Experimental Setups

Here, we provide experimental setups for each analysis task supported by TSLib. For long-term forecasting and zero-shot forecasting, we use a look-back length of 36 and prediction lengths of $\{24, 36, 48, 60\}$ for the ILI dataset. For other datasets, a unified look-back length of 96 is adopted, with forecast horizons of $\{96, 192, 336, 720\}$. In short-term forecasting, the prediction lengths are set as $\{6, 8, 18, 13, 14, 48\}$ for yearly, quarterly, monthly, weekly, daily, and hourly data, respectively. In the classification task, we aim to categorize high-dimensional time series data into multiple classes. In the imputation task, we randomly mask the time points in ratios of $\{12.5\%, 25\%, 37.5\%, 50\%\}$. In the anomaly detection task, the reconstruction is a classical task for unsupervised point-wise representation learning, where the reconstruction error is a natural anomaly criterion. Therefore, we set deep time series models as the base models for reconstruction and utilize the reconstruction error as a shared anomaly criterion across all experiments.

Implementation Details In this paper, all experiments were implemented using PyTorch and executed on an NVIDIA A100 GPU. The Adam optimizer was used for model optimization. Each model's performance was comprehensively evaluated across various time series analysis tasks. To achieve optimal performance and ensure fair comparisons across models and datasets, we conducted hyperparameter searches for each model architecture following [188]. Specifically, we search the number of hidden layers $L \in \{2, 3, 4\}$ and the dimension of hidden representations d_{model} from the range $[2^4, 2^9]$. For each time series analysis task, TSLib utilized task-specific and model-shared training configurations, including learning rate, loss function, batch size, and number of epochs, which are listed in Table 4.

TABLE 4
The default training configuration for each time series analysis task in Time Series Library. LR* denotes the initial learning rate.

Task	LR*	Loss	BatchSize	Epochs
Long-term Forecasting	10^{-4}	MSE	32	10
Short-term Forecasting	10^{-3}	SMAPE	16	10
Imputation	10^{-3}	MSE	16	10
Classification	10^{-3}	Cross Entropy	16	100
Anomaly Detection	10^{-4}	MSE	128	10

Look-back Searching Setting Note that in long-term forecasting tasks, the model's performance is highly dependent on the historical length of the input series. Given the inconsistent look-back length of various papers, we further conducted an additional set of experiments referred to as "Look-back Searching" to ensure a fair and reliable comparison. In these experiments, we systematically searched for input length among $\{96, 192, 336, 512\}$ following [188] and performed an extensive hyperparameter search on each model's architecture to reach their optimal performance. This approach allowed us to assess the maximum achievable performance of each model while ensuring that the reported results accurately reflect their capabilities and facilitate a fair comparison across different models.

6.5 Results and Analyses

To examine the strengths and limitations of various methods in mainstream time series analysis tasks, we select 16 repre-

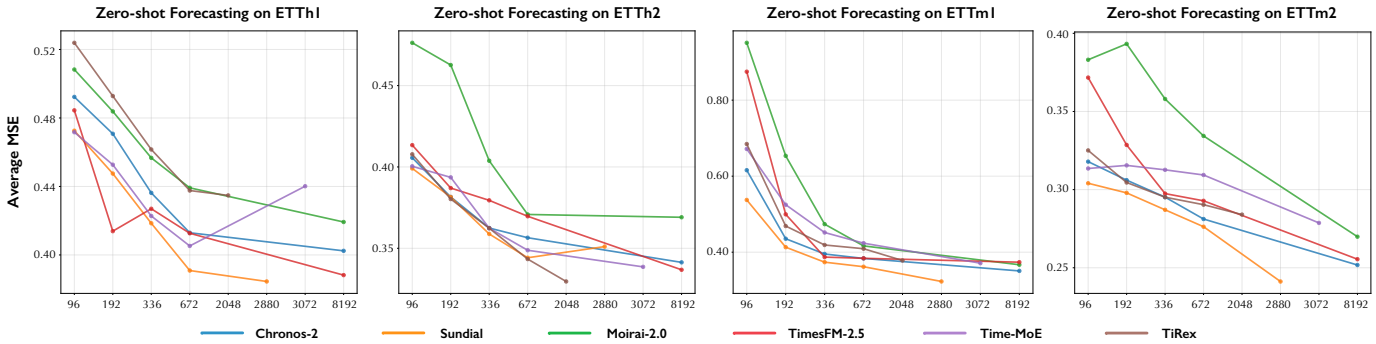


Fig. 11. Zero-shot forecasting performance of different TSFMs with varying input lengths. For Moirai and TimesFM, whose maximum supported context exceeds the available dataset size, we align their input lengths with the maximum capacity of Chronos-2 to ensure a consistent and fair comparison.

sentative deep models and 6 time series foundation models from TSLib for comparison. These models encompass five popular deep model architectures and tasks, including long-term and short-term forecasting, classification, imputation, and anomaly detection.

Baselines To thoroughly explore the effectiveness of different model architectures in various tasks, we conduct comparative experiments using state-of-the-art models designed based on different deep architectures. As shown in Figure 10, we select several advanced and representative Transformer-based models: iTransformer [39], PatchTST [23], Autoformer [22], Non-Stationary Transformer (Stationary) [38], and FEDformer [81] to compare the performance. Additionally, we consider TimesNet [3] and SCINet [127] as the CNN-based models. For RNN-based models, we included the efficient Mamba [116] and representative Neural ODE model, ODERNN [118]. The GNN-based baselines consist of StemGNN [85], MSGNet [189], and TimeFilter [190], all of which enable a data-driven construction of dependency graphs. DLinear [49], N-BEATS [24], and TiDE [94] are tested as representative MLP-based models. Since TiDE [94] depends on timestamps and cannot be easily adapted to tasks without timestamps in their datasets, such as short-term forecasting, anomaly detection, and classification, we only test it on the long-term forecasting task. We also include 6 mainstream time series foundation models, Moirai-2.0 [162], TimesFM-2.5 [166], Time-MoE [167], TiRex [164], Sundial [168], and Chronos-2 [172] to evaluate their zero-shot forecasting performance. Since the ECL, Weather, and Traffic datasets were leaked during pre-training of TimesFM and other TSFMs, the evaluation was limited to the ETT datasets.

Quantitative Results As illustrated in Figure 10, Transformer-based models, notably iTransformer [39] and PatchTST [23], achieve state-of-the-art performance across diverse tasks, particularly in long-term forecasting, which underscores the critical role of specialized embedding and attention mechanisms in unlocking their architectural capacity. GNN-based models also demonstrate competitive efficacy, with TimeFilter [190] and MSGNet [189] excelling in forecasting and anomaly detection respectively, while TimesNet [3] stands out as a versatile milestone for task-general modeling. From an architectural perspective, the results reveal distinct trade-offs: MLP-based models offer surprising efficiency in forecasting but struggle with high-level representation learning, whereas CNN-based architectures exhibit more comprehensive capabilities across non-forecasting tasks. Although GNNs effectively capture inter-series correlations, they generally lack the task-general superiority of leading CNN frameworks. Meanwhile, RNN-based models remain viable for short-term tasks but show limited effectiveness in long-term predictions. Overall, the consistent superiority of Transformers across the benchmark highlights their formidable modeling capacity and potential as a unified backbone for advanced time series analysis.

Under the TSLib protocol, we also include a comparison of the zero-shot forecasting performance between the time series foundation models under varying input lengths. Specifically, we increase the input length from the conventional 96 to the maximum supported context of each model. Figure 11 reports the averaged results across four prediction horizons $\{96, 192, 336, 720\}$ on the ETT datasets. Overall, increasing the input context length generally yields improved zero-shot performance, underscoring the effectiveness of TSFMs in handling long contextual information. Notably, we observe that the performance of Sundial with an input length of 2880 surpasses that of other TSFMs at 8192. This suggests that excessively long inputs may introduce irrelevant information or stochastic noise that obscures meaningful temporal patterns. These findings indicate that optimal zero-shot performance is jointly determined by the model's architectural capacity and the inherent predictability of the input data, underscoring the importance of selecting an appropriate look-back window to balance context richness with data quality.

Qualitative Analysis We also provide an efficiency comparison to offer deeper insights into the practicality and feasibility of different approaches. Specifically, we measure the training time, inference time, and memory usage of each model in performing long-term forecasting, where all models operate under unified hyperparameter settings. As shown in Figure 12 (Right), it is observed that the tokenization design affects the model efficiency, where Autoformer [22] with point-wise tokens is slower than PatchTST [23] with patch-wise tokens and iTransformer [39] with series-wise tokens. Besides, CNN-based models, such as SCINet [127] and TimesNet [3], are much more time-consuming compared to other architectures since they have to adopt diverse kernels to capture intricate temporal patterns. In contrast, MLP-based models and the latest Mamba model stand out for their superior efficiency. Their simpler and more lightweight architectures make them particularly suitable for scenarios with limited computational resources. The above findings underscore the inherent trade-offs between model efficiency

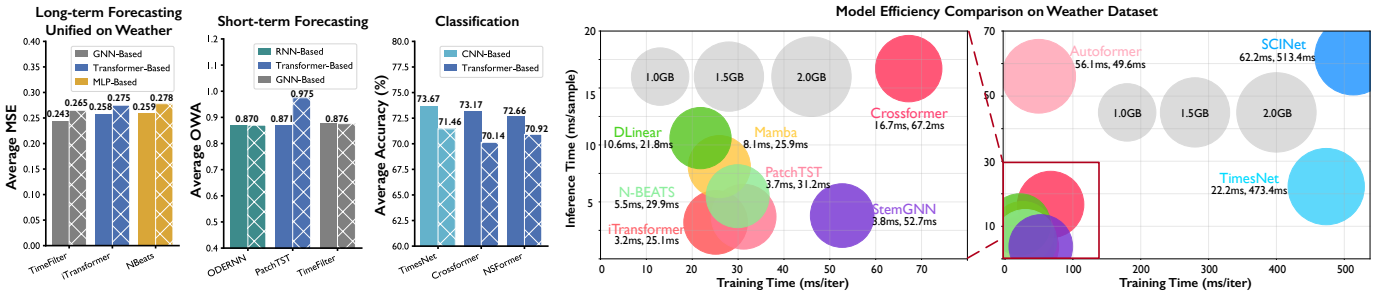


Fig. 12. (Left) Robustness analysis on deep time series models, the cross-hatched bar indicates the performance under noisy data. (Right) Model efficiency tested for long-term forecasting on the Weather dataset.

and predictive performance, which could also be a valuable guide for selecting models for resource-constrained scenarios.

To further assess the practical applicability of the surveyed models, we conducted a robustness analysis on the deep models. Given the inherent complexity and noise in real-world time series data, we systematically added random white noise to the input data to rigorously evaluate model robustness. Specifically, we focused on three representative analysis tasks: long-term forecasting, short-term forecasting, and classification, selecting the top three performing deep models for each task. As shown in Figure 12 (Left),the introduction of noise leads to only marginal performance degradation across most models. This relative stability suggests that these models possess inherent robustness, confirming their viability and reliability in practical scenarios.

7 Future Directions

In this section, we present a discussion on promising directions for time series analysis.

7.1 Time Series Pre-training

The pretraining-finetuning paradigm, widely adopted in language [16], [191] and vision [192], [193], establishes foundational abilities through unsupervised pre-training before task-specific fine-tuning. Given the scarcity of labels, self-supervised pre-training [193] has emerged as a powerful strategy for time series, primarily categorized into contrastive learning [194] and masked modeling [16], [195].

Contrastive learning regularizes representations by contrasting similar and dissimilar pairs [194], [196]. To address the limitations of CV-based methods like SimCLR [196] in temporal modeling, specific approaches have been proposed: CPC [197] captures temporal structure via predictive coding, TS2Vec [198] employs hierarchical instance- and patch-wise losses, and TS-TCC [199] introduces time series-specific augmentations. For long-term dependencies, CoST [200] learns disentangled seasonal-trend representations. Masked modeling is a reconstruction-based approach that predicts masked tokens from surrounding context [16], [195], [201]. Following the BERT paradigm [16], TST [202] first applied this to multivariate time series. Subsequent refinements focus on hierarchical and structural improvements: PatchTST [23] operates at the patch level, while HiMTM [203] utilizes a hierarchical framework for multi-scale characteristics. Additionally, SimMTM [204] integrates manifold learning via neighborhood aggregation, and TimeSiam [205] adopts a Siamese network with asymmetric masking to capture intrinsic temporal correlations.

7.2 Practical Applications

7.2.1 Probabilistic Forecasting

Probabilistic forecasting is a pivotal advancement in practical applications of deep time series models, particularly for informed decision-making. Unlike previous widely-studied deterministic forecasting, probabilistic forecasting offers a distribution over potential future outcomes, enabling stakeholders to assess risks, evaluate confidence intervals, and make robust decisions under uncertainty. Most existing probabilistic forecasting models rely on predefined probability distributions and are designed to predict the parameters of the distributions in the future. While these methods have achieved significant success, the dependence on predefined distributions can limit their ability to represent complex and heterogeneous real-world uncertainties. Therefore, accurately modeling and representing such complexities to generate more flexible and robust probabilistic forecasts remains a critical and active area of future research.

7.2.2 Handling Extremely Long Series

Deep time series models have demonstrated remarkable performance across a wide range of downstream tasks, but their applicability to longer time series data is often limited by scalability and high computational complexity. In industrial time series analysis, high-frequency sampling results in lengthy historical data, impeding the practical implementation of advanced deep models. Existing methods usually include patching techniques to enable them to handle long sequences, and when the input length becomes longer, the patch length can be increased accordingly to reduce the computational complexity. However, model performance is closely tied to patch length; hence, solely increasing patch size to reduce complexity may compromise capabilities. Therefore, addressing the limitations of deep models in handling longer time series could be a promising topic.

7.2.3 Utilizing Exogenous Variables

Since variations within the time series are often influenced by external factors, it is crucial to include exogenous variables in the analysis to gain a more comprehensive understanding of these factors. Exogenous variables, which are widely discussed in time series prediction tasks, are included in the model for uniform training in modeling time series data, without requiring separate analysis. However, in practical applications, different from multivariate time series analysis, the main variables and covariates usually occupy different positions. Given the crucial role played by exogenous variables in real-world applications, it is essential to explore a

unified framework for modeling the relationships between the endogenous and exogenous variables, which allows for a more comprehensive understanding of interrelations and causality among different variants, leading to better and more reliable model performance, as well as interpretability.

7.2.4 Processing Heterogeneous Data

Modeling heterogeneous time series remains a significant challenge. This type of data exhibits diverse characteristics, including varying sampling rates, irregularities, and different length scales, which complicate the effective capture of underlying patterns. Furthermore, the requirement for fixed-size inputs in current deep learning models inherently limits their capability to handle this dynamic, variable-length nature. Addressing these issues demands innovative approaches that can both adapt to individual time series' unique features and capture overarching patterns across multiple series. This includes developing new feature extraction techniques and exploring alternative architectures suitable for variable-length inputs. Further research in this relatively unexplored area holds great promise for significant advances in fields like finance and healthcare, leading to deeper insights and enhanced predictive analytics capabilities.

7.2.5 Handling Complex Disturbed Data

Model robustness is essential for the reliable deployment of deep time series models in real-world applications. In practice, data is often subject to complex disturbances that go beyond typical stochastic fluctuations, including impulse spikes, non-stationary noise, and irregular missing values. Such disturbances can result from sensor instabilities, environmental changes, or intermittent transmission failures, creating highly dynamic and unpredictable data streams. The presence of these intricate disturbances often introduces transient distortions that can obscure underlying temporal patterns. Consequently, addressing these challenging data conditions requires the development of specialized modules designed to handle complex disturbances, making this a critical and promising direction for research. Progress in this area will not only improve the fundamental stability of time series analysis but also enable more dependable decision-making in safety-critical industrial systems.

8 Conclusion

In this survey, we provide a systematic review of deep models in time series analysis and introduce Time-Series Library (TSLib) as a fair benchmark for deep time series models across various tasks. Compared with previous reviews that focus on a specific analysis task or model architecture, this paper provides a comprehensive survey and overview of existing deep models for time series analysis, ranging from forecasting, classification, imputation, and anomaly detection. By separating basic modules from complicated and diverse deep time series models, we successfully uncover the fundamental design principles for current research, forming an inspiring model-centric perspective. Besides, based on TSLib, which enables flexible and comprehensive model evaluation, we conduct extensive experiments to bring insights for model design and practical usage. With clear summarization, rich experiments, and in-depth discussion, we believe that this survey and TSLib can be instructive and useful for future research on deep time series models.

9 Acknowledgments

This work was supported by the National Natural Science Foundation of China (U2342217), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM803), and the National Engineering Research Center for Big Data Software.

References

[1] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time series analysis: forecasting and control*, 2015.
[2] J. D. Hamilton, *Time series analysis*, 2020.
[3] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2d-variation modeling for general time series analysis," in *ICLR*, 2023.
[4] K. Jin, J. Wi, E. Lee, S. Kang, S. Kim, and Y. Kim, "Trafficbert: Pre-trained model with large-scale data for long-range traffic flow forecasting," *Expert Syst Appl.*, 2021.
[5] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *ICLR*, 2022.
[6] H. Wu, H. Zhou, M. Long, and J. Wang, "Interpretable weather forecasting for worldwide stations with a unified deep model," *Nat. Mach. Intell.*, 2023.
[7] L. H. Koopmans, *The spectral analysis of time series*, 1995.
[8] H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: a review," *Data mining and knowledge discovery*, 2019.
[9] M. Braei and S. Wagner, "Anomaly detection in univariate time-series: A survey on the state-of-the-art," *arXiv preprint arXiv:2004.00433*, 2020.
[10] J. F. Torres, D. Hadjout, A. Sebaa, F. Martínez-Álvarez, and A. Troncoso, "Deep learning for time series forecasting: a survey," *Big Data*, 2021.
[11] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *Computing Surveys*, 2021.
[12] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, "Transformers in time series: A survey," *IJCAI*, 2023.
[13] M. Jin, H. Y. Koh, Q. Wen, D. Zambon, C. Alippi, G. I. Webb, I. King, and S. Pan, "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *TPAMI*, 2024.
[14] Z. Shao, F. Wang, Y. Xu, W. Wei, C. Yu, Z. Zhang, D. Yao, G. Jin, X. Cao, G. Cong *et al.*, "Exploring progress in multivariate time series forecasting: Comprehensive benchmarking and heterogeneity analysis," *TKDE*, 2024.
[15] X. Qiu, J. Hu, L. Zhou, X. Wu, J. Du, B. Zhang, C. Guo, A. Zhou, C. S. Jensen, Z. Sheng *et al.*, "Tfb: Towards comprehensive and fair benchmarking of time series forecasting methods," *VLDB*, 2024.
[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *ACL*, 2019.
[17] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.
[19] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.
[20] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *Computing surveys*, 2019.
[21] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *AAAI*, 2021.
[22] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *NeurIPS*, 2021.

[23] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *ICLR*, 2023.

[24] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: neural basis expansion analysis for interpretable time series forecasting," in *ICLR*, 2020.

[25] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel recurrent neural networks," in *ICML*, 2016.

[26] S. Oprea, P. Martinez-Gonzalez, A. Garcia-Garcia, J. A. Castro-Vargas, S. Orts-Escolano, J. Garcia-Rodriguez, and A. Argyros, "A review on deep learning techniques for video prediction," *TPAMI*, 2020.

[27] J. H. Stock and M. W. Watson, "Vector autoregressions," *J. Econ. Perspect.*, 2001.

[28] A. S. Weigend, *Time series prediction: forecasting the future and understanding the past*, 2018.

[29] G. P. Zhang, "Time series forecasting using a hybrid arima and neural network model," *Neurocomputing*, 2003.

[30] C. Fang and C. Wang, "Time series data imputation: A survey on deep learning approaches," *arXiv preprint arXiv:2011.11347*, 2020.

[31] Y. Luo, X. Cai, Y. Zhang, J. Xu *et al.*, "Multivariate time series imputation with generative adversarial networks," in *NeurIPS*, 2018.

[32] S. Moritz and T. Bartz-Beielstein, "imputets: time series missing value imputation in r." *R Journal*, 2017.

[33] B. Zhao, H. Lu, S. Chen, J. Liu, and D. Wu, "Convolutional neural networks for time series classification," *J. Syst. Eng. Electron.*, 2017.

[34] N. Laptev, S. Amizadeh, and I. Flint, "Generic and scalable framework for automated time-series anomaly detection," in *KDD*, 2015.

[35] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," *arXiv preprint arXiv:1607.08022*, 2016.

[36] N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Deep adaptive input normalization for time series forecasting," *TNNLS*, 2019.

[37] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *ICLR*, 2021.

[38] Y. Liu, H. Wu, J. Wang, and M. Long, "Non-stationary transformers: Exploring the stationarity in time series forecasting," in *NeurIPS*, 2022.

[39] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "itransformer: Inverted transformers are effective for time series forecasting," *ICLR*, 2024.

[40] J. Gao, W. Hu, and Y. Chen, "Client: Cross-variable linear integrated enhanced transformer for multivariate long-term time series forecasting," *AI Open*, 2025.

[41] Z. Liu, M. Cheng, Z. Li, Z. Huang, Q. Liu, Y. Xie, and E. Chen, "Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective," in *NeurIPS*, 2024.

[42] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "Stl: A seasonal-trend decomposition," *J. Off. Stat*, 1990.

[43] O. D. Anderson, "Time-series. 2nd edn." 1976.

[44] C. RB, "Stl: A seasonal-trend decomposition procedure based on loess," *J. Off. Stat.*, 1990.

[45] E. B. Dagum and S. Bianconcini, *Seasonal adjustment methods and real time trend-cycle estimation*. Springer, 2016.

[46] Q. Wen, J. Gao, X. Song, L. Sun, H. Xu, and S. Zhu, "Robuststl: A robust seasonal-trend decomposition algorithm for long time series," in *AAAI*, 2019.

[47] A. M. De Livera, R. J. Hyndman, and R. D. Snyder, "Forecasting time series with complex seasonal patterns using exponential smoothing," *J. Am. Stat. Assoc.*, 2011.

[48] S. J. Taylor and B. Letham, "Forecasting at scale," *Am. Stat.*, 2018.

[49] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *AAAI*, 2023.

[50] H. Cao, Z. Huang, T. Yao, J. Wang, H. He, and Y. Wang, "Inparformer: evolutionary decomposition transformers with interactive parallel attention for long-term time series forecasting," in *AAAI*, 2023.

[51] D. Du, B. Su, and Z. Wei, "Preformer: predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting," in *ICASSP*, 2023.

[52] D. Cao, F. Jia, S. O. Arik, T. Pfister, Y. Zheng, W. Ye, and Y. Liu, "Tempo: Prompt-based generative pre-trained transformer for time series forecasting," *ICLR*, 2024.

[53] C. Challu, K. G. Olivares, B. N. Oreshkin, F. G. Ramirez, M. M. Canseco, and A. Dubrawski, "Nhits: Neural hierarchical interpolation for time series forecasting," in *AAAI*, 2023.

[54] W. Fan, S. Zheng, X. Yi, W. Cao, Y. Fu, J. Bian, and T.-Y. Liu, "Depts: Deep expansion learning for periodic time series forecasting," in *ICLR*, 2022.

[55] W. Fan, Y. Fu, S. Zheng, J. Bian, Y. Zhou, and H. Xiong, "Dewp: Deep expansion learning for wind power forecasting," *TKDD*, 2024.

[56] L. Xiong, X. Chen, T.-K. Huang, J. Schneider, and J. G. Carbonell, "Temporal collaborative filtering with bayesian probabilistic tensor factorization," in *SIAM-SDM*, 2010.

[57] H.-F. Yu, N. Rao, and I. S. Dhillon, "Temporal regularized matrix factorization for high-dimensional time series prediction," in *NeurIPS*, 2016.

[58] K. Takeuchi, H. Kashima, and N. Ueda, "Autoregressive tensor factorization for spatio-temporal predictions," in *ICDM*, 2017.

[59] X. Chen, C. Zhang, X.-L. Zhao, N. Saunier, and L. Sun, "Nonstationary temporal matrix factorization for multivariate time series forecasting," *arXiv preprint arXiv:2203.10651*, 2022.

[60] X. Chen and L. Sun, "Bayesian temporal factorization for multidimensional time series prediction," *TPAMI*, 2021.

[61] R. Sen, H.-F. Yu, and I. S. Dhillon, "Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting," in *NeurIPS*, 2019.

[62] J.-M. Yang, Z.-R. Peng, and L. Lin, "Real-time spatiotemporal prediction and imputation of traffic status based on lstm and graph laplacian regularized matrix factorization," *Transp. Res. Part C Emerg. Technol.*, 2021.

[63] D. Cai, X. He, J. Han, and T. S. Huang, "Graph regularized nonnegative matrix factorization for data representation," *TPAMI*, 2010.

[64] I. N. Sneddon, *Fourier transforms*. Courier Corporation, 1995.

[65] J. W. Goodman, *Introduction to Fourier optics*. Roberts and Company publishers, 2005.

[66] P. Bloomfield, *Fourier analysis of time series: an introduction*. John Wiley & Sons, 2004.

[67] K. Samiee, P. Kovacs, and M. Gabbouj, "Epileptic seizure classification of eeg time-series using rational discrete short-time fourier transform," *TBME*, 2014.

[68] E. O. Brigham, *The fast Fourier transform and its applications*. Prentice-Hall, Inc., 1988.

[69] Y. Meyer, *Wavelets and operators: volume 1*. Cambridge university press, 1992.

[70] J. Wang, Z. Wang, J. Li, and J. Wu, "Multilevel wavelet decomposition network for interpretable time series analysis," in *KDD*, 2018.

[71] S. Yao, A. Piao, W. Jiang, Y. Zhao, H. Shao, S. Liu, D. Liu, J. Li, T. Wang, S. Hu *et al.*, "Stfnets: Learning sensing signals from the time-frequency perspective with short-time fourier neural networks," in *WWW*, 2019.

[72] M. Rhif, A. Ben Abbes, I. R. Farah, B. Martínez, and Y. Sang, "Wavelet transform application for/in non-stationary time-series analysis: A review," *Applied Sciences*, 2019.

[73] L. Minhao, A. Zeng, L. Qiuxia, R. Gao, M. Li, J. Qin, and Q. Xu, "T-wavenet: A tree-structured wavelet neural network for time series signal analysis," in *ICLR*, 2021.

[74] L. Yang and S. Hong, "Unsupervised time-series representation learning with iterative bilinear temporal-spectral fusion," in *ICML*, 2022.

[75] J. Wang, C. Yang, X. Jiang, and J. Wu, "When: A wavelet-dtw hybrid attention network for heterogeneous time series analysis," in *KDD*, 2023.

[76] F. Yang, X. Li, M. Wang, H. Zang, W. Pang, and M. Wang, "Waveform: Graph enhanced wavelet learning for long sequence forecasting of multivariate time series," in *AAAI*, 2023.

[77] K. Yi, Q. Zhang, L. Cao, S. Wang, G. Long, L. Hu, H. He, Z. Niu, W. Fan, and H. Xiong, "A survey on deep learning based time series analysis with frequency transformation," *KDD*, 2025.

[78] T. Dai, B. Wu, P. Liu, N. Li, J. Bao, Y. Jiang, and S.-T. Xia, "Periodicity decoupling framework for long-term series forecasting," in *ICLR*, 2023.

[79] T. Zhou, Z. Ma, Q. Wen, L. Sun, T. Yao, W. Yin, R. Jin *et al.*, "Film: Frequency improved legendre memory model for long-term time series forecasting," in *NeurIPS*, 2022.

[80] Z. Xu, A. Zeng, and Q. Xu, "Fits: Modeling time series with $10k$ parameters," *ICLR*, 2024.

[81] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *ICML*, 2022.

[82] N. Wiener, "Generalized harmonic analysis," *Acta mathematica*, 1930.

[83] Z. Yang, W. Yan, X. Huang, and L. Mei, "Adaptive temporal-frequency network for time-series forecasting," *TKDE*, 2020.

[84] C. Zhang, T. Zhou, Q. Wen, and L. Sun, "Tfad: A decomposition time series anomaly detection architecture with time-frequency analysis," in *CIKM*, 2022.

[85] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong *et al.*, "Spectral temporal graph neural network for multivariate time-series forecasting," in *NeurIPS*, 2020.

[86] K. Yi, Q. Zhang, L. Hu, H. He, N. An, L. Cao, and Z. Niu, "Edge-varying fourier graph networks for multivariate time series forecasting," *arXiv preprint arXiv:2210.03093*, 2022.

[87] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, N. An, D. Lian, L. Cao, and Z. Niu, "Frequency-domain mlps are more effective learners in time series forecasting," in *NeurIPS*, 2024.

[88] Z. Wang, C. Pei, M. Ma, X. Wang, Z. Li, D. Pei, S. Rajmohan, D. Zhang, Q. Lin, H. Zhang *et al.*, "Revisiting vae for unsupervised time series anomaly detection: A frequency perspective," in *WWW*, 2024.

[89] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "Tslanet: Rethinking transformers for time series representation learning," in *ICML*, 2024.

[90] J. Crabbé, N. Huynh, J. Stanczuk, and M. van der Schaar, "Time series diffusion in the frequency domain," in *ICML*, 2024.

[91] K. G. Olivares, C. Challu, G. Marcjasz, R. Weron, and A. Dubrawski, "Neural basis expansion analysis with exogenous variables: Forecasting electricity prices with nbeatsx," *Int. J. Forecast.*, 2023.

[92] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit *et al.*, "Mlp-mixer: An all-mlp architecture for vision," in *NeurIPS*, 2021.

[93] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, "Tsmixer: An all-mlp architecture for time series forecasting," *TMLR*, 2023.

[94] A. Das, W. Kong, A. Leach, R. Sen, and R. Yu, "Long-term forecasting with tide: Time-series dense encoder," *TMLR*, 2023.

[95] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, 2010.

[96] Y. Liu, C. Li, J. Wang, and M. Long, "Koopa: Learning non-stationary time series dynamics with koopman predictors," *NeurIPS*, 2023.

[97] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, 1997.

[98] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *NeurIPS*, 2014.

[99] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *EMNLP*, 2014.

[100] R. Nallapati, B. Zhou, C. Gulcehre, and B. Xiang, "Abstractive text summarization using sequence-to-sequence rnns and beyond," in *SIGNLL*, 2016.

[101] S. Mehri, K. Kumar, I. Gulrajani, R. Kumar, S. Jain, J. Sotelo, A. Courville, and Y. Bengio, "Samplernn: An unconditional end-to-end neural audio generation model," *ICLR*, 2017.

[102] A. Tealab, "Time series forecasting using artificial neural networks methodologies: A systematic review," *Future Computing and Informatics Journal*, 2018.

[103] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, "Brits: Bidirectional recurrent imputation for time series," in *NeurIPS*, 2018.

[104] J. Yoon, W. R. Zame, and M. van der Schaar, "Estimating missing data in temporal data streams using multi-directional recurrent neural networks," *TBME*, 2018.

[105] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific reports*, 2018.

[106] I. M. Baytas, C. Xiao, X. Zhang, F. Wang, A. K. Jain, and J. Zhou, "Patient subtyping via time-aware lstm networks," in *KDD*, 2017.

[107] L. Shen, Z. Li, and J. Kwok, "Timeseries anomaly detection using temporal hierarchical one-class network," in *NeurIPS*, 2020.

[108] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long-and short-term temporal patterns with deep neural networks," in *SIGIR*, 2018.

[109] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. Cottrell, "A dual-stage attention-based recurrent neural network for time series prediction," *IJCAI*, 2017.

[110] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "Deepar: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecast.*, 2020.

[111] A. Gu, I. Johnson, K. Goel, K. Saab, T. Dao, A. Rudra, and C. Ré, "Combining recurrent, convolutional, and continuous-time models with linear state space layers," in *NeurIPS*, 2021.

[112] M. Fraccaro, S. K. Sønderby, U. Paquet, and O. Winther, "Sequential neural models with stochastic layers," in *NeurIPS*, 2016.

[113] R. Krishnan, U. Shalit, and D. Sontag, "Structured inference networks for nonlinear state space models," in *AAAI*, 2017.

[114] S. S. Rangapuram, M. W. Seeger, J. Gasthaus, L. Stella, Y. Wang, and T. Januschowski, "Deep state space models for time series forecasting," in *NeurIPS*, 2018.

[115] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *ICLR*, 2022.

[116] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *COLM*, 2024.

[117] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," *NeurIPS*, 2018.

[118] Y. Rubanova, R. T. Chen, and D. K. Duvenaud, "Latent ordinary differential equations for irregularly-sampled time series," *NeurIPS*, 2019.

[119] P. Kidger, J. Morrill, J. Foster, and T. Lyons, "Neural controlled differential equations for irregular time series," *NeurIPS*, 2020.

[120] Y. Yin, V. Le Guen, J. Dona, E. De Bézenac, I. Ayed, N. Thome, and P. Gallinari, "Augmenting physical models with deep networks for complex dynamics forecasting," *J Stat Mech-theory E*, 2021.

[121] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai *et al.*, "Recent advances in convolutional neural networks," *Pattern recognition*, 2018.

[122] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *ICML*, 2019.

[123] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," *TPAMI*, 2021.

[124] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *CVPR*, 2016.

[125] A. Le Guennec, S. Malinowski, and R. Tavenard, "Data augmentation for time series classification using convolutional neural networks," in *ECML/PKDD workshop*, 2016.

[126] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "Inceptiontime: Finding alexnet for time series classification," *Data Mining and Knowledge Discovery*, 2020.

[127] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, and Q. Xu, "Scinet: Time series modeling and forecasting with sample convolution and interaction," in *NeurIPS*, 2022.

[128] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "Wavenet: A generative model for raw audio," *Proc. SSW*, 2016.

[129] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[130] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, and Y. Xiao, "Micn: Multi-scale local and global context modeling for long-term series forecasting," in *ICLR*, 2022.

[131] D. Luo and X. Wang, "Moderntcn: A modern pure convolution structure for general time series analysis," in *ICLR*, 2024.

[132] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *CVPR*, 2015.

[133] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *CVPR*, 2016.

[134] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *TNNLS*, 2008.

[135] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[136] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *ICLR*, 2018.

[137] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *IJCAI*, 2018.

[138] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *IJCAI*, 2019.

[139] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *NeurIPS*, 2020.

[140] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *KDD*, 2020.

[141] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," in *AAAI*, 2021.

[142] J. Lu, D. Batra, D. Parikh, and S. Lee, "Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks," in *NeurIPS*, 2019.

[143] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le, "Xlnet: Generalized autoregressive pretraining for language understanding," in *NeurIPS*, 2019.

[144] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[145] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[146] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *ICML*, 2021.

[147] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *ICCV*, 2021.

[148] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," in *ICCV*, 2021.

[149] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[150] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *NeurIPS*, 2019.

[151] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A. X. Liu, and S. Dustdar, "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *ICLR*, 2021.

[152] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *ICLR*, 2023.

[153] T. Zhou, P. Niu, L. Sun, R. Jin *et al.*, "One fits all: Power general time series analysis by pretrained lm," in *NeurIPS*, 2024.

[154] Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long, "Timer: Transformers for time series analysis at scale," *ICML*, 2024.

[155] S. Papoulis, *Probability, Random Variables and Stochastic Processes by Athanasios*. Boston: McGraw-Hill, 2002.

[156] W. Xue, T. Zhou, Q. Wen, J. Gao, B. Ding, and R. Jin, "Card: Channel aligned robust blend transformer for time series forecasting," in *ICLR*, 2024.

[157] P. Chen, Y. Zhang, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, B. Yang, and C. Guo, "Multi-scale transformers with adaptive pathways for time series forecasting," in *ICLR*, 2023.

[158] Y. Wang, H. Wu, J. Dong, Y. Liu, Y. Qiu, H. Zhang, J. Wang, and M. Long, "Timexer: Empowering transformers for time series forecasting with exogenous variables," in *NeurIPS*, 2024.

[159] S. Dooley, G. S. Khurana, C. Mohapatra, S. V. Naidu, and C. White, "Forecastpfn: Synthetically-trained zero-shot forecasting," in *NeurIPS*, 2023.

[160] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, and A. Dubrawski, "Moment: A family of open time-series foundation models," in *ICML*, 2024.

[161] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," *ICML*, 2024.

[162] X. Liu, J. Liu, G. Woo, T. Aksu, Y. Liang, R. Zimmermann, C. Liu, S. Savarese, C. Xiong, and D. Sahoo, "Moirai-moe: Empowering time series foundation models with sparse mixture of experts," *ICML*, 2025.

[163] V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. Nguyen, W. M. Gifford, C. Reddy, and J. Kalagnanam, "Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series," in *NeurIPS*, 2024.

[164] A. Auer, P. Podest, D. Klotz, S. Böck, G. Klambauer, and S. Hochreiter, "Tirex: Zero-shot forecasting across long and short horizons with enhanced in-context learning," *NeurIPS*, 2025.

[165] K. Rasul, A. Ashok, A. R. Williams, A. Khorasani, G. Adamopoulos, R. Bhagwatkar, M. Biloš, H. Ghonia, N. Hassen, A. Schneider *et al.*, "Lag-llama: Towards foundation models for time series forecasting," in *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*, 2023.

[166] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *ICML*, 2024.

[167] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, and M. Jin, "Time-moe: Billion-scale time series foundation models with mixture of experts," in *ICLR*, 2025.

[168] Y. Liu, G. Qin, Z. Shi, Z. Chen, C. Yang, X. Huang, J. Wang, and M. Long, "Sundial: A family of highly capable time series foundation models," *ICML*, 2025.

[169] B. Cohen, E. Khwaja, K. Wang, C. Masson, E. Ramé, Y. Doubli, and O. Abou-Amal, "Toto: Time series optimized transformer for observability," *arXiv preprint arXiv:2407.07874*, 2024.

[170] S. Q. Nate Gruver, Marc Finzi and A. G. Wilson, "Large Language Models Are Zero Shot Time Series Forecasters," in *NeurIPS*, 2023.

[171] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. P. Arango, S. Kapoor *et al.*, "Chronos: Learning the language of time series," *TMLR*, 2024.

[172] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado, S. S. Rangapuram, H. Shen, L. Stella, X. Zhang *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.

[173] F. Li, J. Feng, H. Yan, G. Jin, F. Yang, F. Sun, D. Jin, and Y. Li, "Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution," *TKDD*, 2023.

[174] J. Wang, J. Jiang, W. Jiang, C. Li, and W. X. Zhao, "Libcity: An open library for traffic prediction," in *SIGSPATIAL*, 2021.

[175] R. Jiang, D. Yin, Z. Wang, Y. Wang, J. Deng, H. Liu, Z. Cai, J. Deng, X. Song, and R. Shibasaki, "Dl-traff: Survey and benchmark of deep learning models for urban traffic prediction," in *CIKM*, 2021.

[176] Y. Hao, X. Qin, Y. Chen, Y. Li, X. Sun, Y. Tao, X. Zhang, and X. Du, "Ts-benchmark: A benchmark for time series databases," in *ICDE*, 2021.

[177] R. Tavenard, J. Faouzi, G. Vandewiele, F. Divo, G. Androz, C. Holtz, M. Payne, R. Yurchak, M. Rußwurm, K. Kolar, and E. Woods, "Tslearn, a machine learning toolkit for time series data," *JMLR*, 2020.

[178] F. Garza, M. M. Canseco, C. Challú, and K. G. Olivares, "StatsForecast: Lightning fast forecasting with statistical and econometric models," 2022. [Online]. Available: https://github.com/Nixtla/statsforecast

[179] K. G. Olivares, C. Challú, A. Garza, M. M. Canseco, and A. Dubrawski, "NeuralForecast: User friendly state-of-the-art neural forecasting models." 2022. [Online]. Available: https://github.com/Nixtla/neuralforecast

[180] A. Alexandrov, K. Benidis, M. Bohlke-Schneider, V. Flunkert, J. Gasthaus, T. Januschowski, D. C. Maddix, S. Rangapuram, D. Salinas, J. Schulz, L. Stella, A. C. Türkmen, and Y. Wang, "GluonTS: Probabilistic and Neural Time Series Modeling in Python," *JMLR*, 2020.

[181] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The m4 competition: 100,000 time series and 61 forecasting methods," *Int. J. Forecast.*, 2020.

[182] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *KDD*, 2019.

[183] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding," in *KDD*, 2018.

[184] A. P. Mathur and N. O. Tippenhauer, "Swat: A water treatment testbed for research and training on ics security," in *CySWater*, 2016.

[185] A. Abdulaal, Z. Liu, and T. Lancewicki, "Practical approach to asynchronous multivariate time series anomaly detection and localization," *KDD*, 2021.

[186] A. J. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. G. Bostrom, P. Southam, and E. J. Keogh, "The uea multivariate time series classification archive, 2018," *arXiv preprint arXiv:1811.00075*, 2018.

[187] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," *KDD*, 2021.

[188] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, and J. ZHOU, "Timemixer: Decomposable multiscale mixing for time series forecasting," in *ICLR*, 2024.
[189] W. Cai, Y. Liang, X. Liu, J. Feng, and Y. Wu, "Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting," in *AAAI*, 2024.
[190] Y. Hu, G. Zhang, P. Liu, D. Lan, N. Li, D. Cheng, T. Dai, S.-T. Xia, and S. Pan, "Timefilter: Patch-specific spatial-temporal graph filtration for time series forecasting," in *ICML*, 2025.
[191] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Sci. China Technol. Sci.*, 2020.
[192] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *CVPR*, 2020.
[193] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *TKDE*, 2021.
[194] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *Technologies*, 2020.
[195] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *CVPR*, 2022.
[196] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *ICML*, 2020.
[197] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.
[198] Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, and B. Xu, "Ts2vec: Towards universal representation of time series," in *AAAI*, 2022.
[199] E. Eldele, M. Ragab, Z. Chen, M. Wu, C. K. Kwoh, X. Li, and C. Guan, "Time-series representation learning via temporal and contextual contrasting," in *IJCAI*, 2021.
[200] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. Hoi, "CoST: Contrastive learning of disentangled seasonal-trend representations for time series forecasting," in *ICLR*, 2022.
[201] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, "Data2vec: A general framework for self-supervised learning in speech, vision and language," in *ICML*, 2022.
[202] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *KDD*, 2021.
[203] S. Zhao, M. Jin, Z. Hou, C. Yang, Z. Li, Q. Wen, and Y. Wang, "Himtm: Hierarchical multi-scale masked time series modeling for long-term forecasting," *CIKM*, 2024.
[204] J. Dong, H. Wu, H. Zhang, L. Zhang, J. Wang, and M. Long, "Simmtm: A simple pre-training framework for masked time-series modeling," *NeurIPS*, 2023.
[205] J. Dong, H. Wu, Y. Wang, Y. Qiu, L. Zhang, J. Wang, and M. Long, "Timesiam: A pre-training framework for siamese time-series modeling," *ICML*, 2024.



Yuxuan Wang received the BE degree from Beihang University in 2022. She is working towards the PhD degree in computer software at Tsinghua University. Her research interests include machine learning and time series analysis.



Haixu Wu received the BE degree in software engineering from Tsinghua University in 2020. He is working towards the PhD degree in computer software at Tsinghua University. His research interests include scientific machine learning, deep learning, and spatiotemporal learning.



Jiaxiang Dong received the ME degree in computer science and technology from Nankai University in 2018. He is currently working toward the PhD degree in computer software at Tsinghua University. His research interests include machine learning and time series analysis.



Yong Liu received the BE degree in software engineering from Tsinghua University in 2021. He is working towards the PhD degree in computer software at Tsinghua University. His research interests include time series analysis and time series foundation models.



Chen Wang received B.S. and M.S. degrees in computer science from Fudan University in 2003 and 2006, respectively. He is currently a research associate professor and CTO at National Engineering Laboratory for Big Data Software, Tsinghua University. His research focuses on database systems, data governance, time series data management, and industrial big data applications



Mingsheng Long received the BE and PhD degrees from Tsinghua University in 2008 and 2014 respectively. He is a tenured associate professor with the School of Software, Tsinghua University. He serves as an associate editor of *IEEE Transactions on Pattern Analysis and Machine Intelligence* and *Artificial Intelligence Journal*, and as senior/highlighted area chairs of major machine learning conferences, including NeurIPS, ICML, and ICLR. His research is dedicated to machine learning theory, algorithms, and models, with special interests in deep learning and foundation models, scientific learning and world models, transfer learning and domain adaptation.



Jianmin Wang received the BE degree from Peking University in 1990, and the ME and PhD degrees in computer software from Tsinghua University in 1992 and 1995, respectively. He is a full professor with the School of Software, Tsinghua University. His research interests include Big Data management systems and large-scale data analytics. He led to developing a product data and lifecycle management system, which has been deployed in hundreds of enterprises in China. He is leading the development of the Tsinghua DataWay Big Data platform and Apache IoTDB time series database systems in the National Engineering Lab for Big Data Software.